

SHIFTWM: WORLD MODELS THAT MOVE WHAT THEY SAW

Anonymous authors

Paper under double-blind review

ABSTRACT

A world model forecasts how a scene responds to actions. Forecasting in the feature space of a frozen visual foundation model yields compact models that support planning without generating pixels. Current latent world models such as DINO-WM and V-JEPA 2-AC regenerate the entire patch-feature grid at every future step and usually feed their own predictions back as inputs. Yet in manipulation and surgical scenes most content persists and only moves. Regenerating it wastes capacity, blurs moving objects, corrupts static background and lets errors compound over the rollout. We introduce SHIFTWM, a world model that forecasts by *moving what it has already seen*. For each prefix of future actions it predicts a local soft transport of measured patch features, a gate that decides where transport applies, and a correction for content that transport cannot explain. Every horizon is decoded directly from observed features, and a simple motion model shows that the MSE-optimal forecast is itself such a transport. We compare SHIFTWM with predictors that share its frozen encoder and architecture on held-out real robot video (DROID, Language-Table) and seven surgical robot tasks, and attach it as a plug-in head to two published world models. On held-out real robot video (DROID), SHIFTWM lowers forecast error by 7.6% relative to a matched direct predictor and by 12.2% relative to a matched recursive one; on block pushing (Language-Table) it lowers error by 6.9% and 8.6%, and on surgical tasks the gains are smaller (3.6% and 4.0%). Added to V-JEPA 2-AC under an identical fine-tuning budget, it lowers forecast error by 15.7%; added to DINO-WM, it lowers latent error by 8.0% on PushT but raises it by 18.4% on Wall.

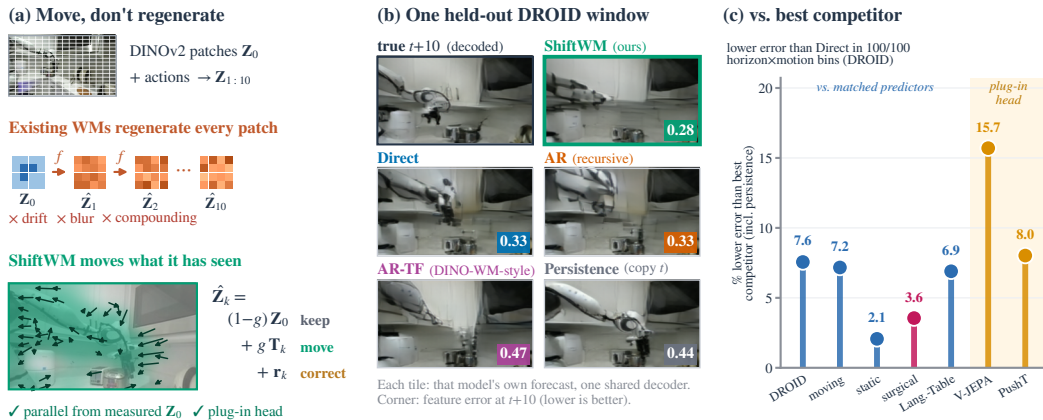


Figure 1: **SHIFTWM moves what it has seen instead of regenerating it.** (b) Each model’s own step-10 forecast of a selected held-out DROID window (rule in Appendix E.3), decoded by one shared decoder. (c) % lower error than the best competitor, persistence included.

1 INTRODUCTION

A world model predicts how observations change in response to actions, so that an agent can evaluate candidate actions before executing them. A growing line of work forecasts in the patch-feature space of a frozen self-supervised encoder such as DINOv2 or V-JEPA 2, which gives compact models that plan without generating pixels (Zhou et al., 2025; Assran et al., 2025; Maes et al., 2026b; Sobal et al., 2025).

Current latent world models are *generative in feature space*: at every future step they regress the complete grid of patch features, and most feed their own predictions back as inputs. In manipulation and surgical video, however, most of the scene persists: the table, tissue, tools and the robot move, but they are rarely created or destroyed. Regenerating persistent content causes three concrete problems. Static background must be rewritten exactly, yet on held-out robot video every matched regression predictor we train is worse than simply copying the last frame on static patches. Moving content must be synthesised as a displacement in feature space, which blurs object boundaries. Recursive rollouts compound these errors: once rolled out, a DINO-WM-style predictor trained with teacher forcing is worse than persistence on held-out robot video. Yet much of the future is already observed: on moving patches of held-out DROID, copying the single best observed feature from a local window removes 51% of the error of staying put. Pixel and feature forecasters exploit such motion by warping (Finn et al., 2016; Liu et al., 2017; Šarić et al., 2020), but latent world models for planning do not.

We introduce SHIFTWLM, a latent world model that forecasts by *moving what it has already seen* (Figures 1 and 2). Instead of regressing the next feature grid, it predicts, for every future step, a motion of the grids it has observed: each patch softly selects where its content comes from within a local window of measured features, a gate decides whether it stays or moves, and a small correction adds what motion cannot explain. The motion is conditioned on the causal prefix of future actions, and every horizon is decoded in parallel from measured features, so predictions are never fed back. The transport weights, gate and correction are explicit tensors that show where the model predicts motion, and because the design is an output head, it can be attached to published world models.

Our evaluation suite has 18 evaluations across four axes (Figure 4): forecasting (DROID and its second camera, BridgeData V2, RT-1, Language-Table, IWS, seven Open-H surgical tasks), plug-in heads (V-JEPA 2-AC, DINO-WM PushT and Wall), closed-loop planning (three suites) and five analyses. This version reports DROID, Language-Table, the surgical tasks, the plug-ins and the analyses of Section 5.3; the rest are marked pending.

Contributions.

- We introduce SHIFTWLM, a latent world model whose output is a learned soft local transport of frozen foundation-model patch features, conditioned on the action prefix, gated against the identity, corrected for new content and decoded in parallel for every horizon. It adds $< 0.3\%$ parameters to a matched predictor (Section 3).
- We demonstrate that SHIFTWLM works as a plug-in head for published world models: it lowers V-JEPA 2-AC’s forecast error by 15.7% with 0.16M added parameters under the same fine-tuning budget, and its gate opens to 56–65%, so the large model routes most of its forecast through transport. It lowers DINO-WM’s latent error by 8.0% on PushT, but raises it by 18.4% on Wall (Section 5.2).
- We show that under a simple motion model the MSE-optimal forecast is exactly a transport of observed features, which transport parameterises directly, whereas an additive head must synthesise the feature contrast along the motion, and we check the premise without a model (Sections 3.5 and 5.3).
- We set up an evaluation suite of 18 evaluations across four axes (forecasting, plug-in, planning, analysis), and demonstrate against predictors with the same encoder and architecture that SHIFTWLM lowers held-out DROID error by 7.6% relative to a direct and 12.2% relative to a recursive predictor, with small gains on seven surgical tasks (3.6%). It has both the lowest error and the highest action-ranking accuracy on DROID (92.0% vs. 89.8% for Direct), and is the only learned predictor below persistence on static DROID patches (Section 5).

2 RELATED WORK

Latent and foundation-feature world models. A recent line of work predicts dynamics in the embedding space of a pretrained or jointly learned encoder instead of in pixels. DINO-WM trains a causal ViT predictor on frozen DINOv2 patch features and plans with model-predictive control (Zhou et al., 2025; Oquab et al., 2024); PLDM, LeWM and JEPA-WMs learn joint-embedding predictive models from offline trajectories (Sobal et al., 2025; Maes et al., 2026b; Balestriero & LeCun, 2025; Terver et al., 2026); and V-JEPA 2-AC post-trains an action-conditioned predictor on DROID for zero-shot robot planning (Assran et al., 2025; Khazatsky et al., 2024). DINO-world, DINO-Foresight, VFMF and FlowWM forecast future foundation features, deterministically or generatively (Baldassarre et al., 2025; Karypidis et al., 2025; Boduljak et al., 2025; Porcher et al., 2026); DDP-WM separates sparse dynamic regions from a static background, RLA-WM models residual latent actions (Yin et al., 2026; Zhang et al., 2026a), and several analyses ask which latent spaces and objectives make predictions action-relevant (Nilaksh et al., 2026; Yeom et al., 2026; Zhang et al., 2026b; Huang et al., 2026). Pixel-space and model-based RL world models are complementary (Bar et al., 2025; NVIDIA, 2025a; Hafner et al., 2025; Hansen et al., 2024) (Appendix A.5). In all latent models above, the predictor synthesises every future feature map, including content that only moves. SHIFTWm keeps the frozen-encoder interface but changes the output parameterisation: each forecast is a transport of observed features plus a correction, and the same head can be attached to these models.

Recursive versus direct multi-step prediction. Iterating a one-step model compounds its errors, a classic observation in multi-step learning (Venkatraman et al., 2015). Most latent world models are nevertheless trained for one step and rolled out recursively. DINO-WM uses teacher forcing and autoregressive rollouts (Zhou et al., 2025), V-JEPA 2-AC adds a rollout loss to teacher-forced training (Assran et al., 2025), and PlaNet introduced multi-step latent overshooting to reduce the mismatch between one-step training and multi-step use (Hafner et al., 2019). Anchored navigation world models resist drift by conditioning on anchor frames (Luan et al., 2026), and decision-oriented objectives align latent predictions with the costs that a planner ranks (Wang et al., 2026a). Direct prediction of every horizon avoids feedback, but a direct regressor must still regenerate the whole grid. SHIFTWm, like our Direct baseline, decodes every horizon from measured features; it differs in *what* it decodes.

Motion-, flow- and feature-warping prediction. Predicting the future by moving existing content has a long history. In pixel space, CDNA and DNA transform the previous frame with action-conditioned kernels or motion distributions (Finn et al., 2016), dynamic filter networks generate input-dependent filters (De Brabandere et al., 2016), Vondrick and Torralba transform past pixels with adversarially trained transformers (Vondrick & Torralba, 2017), SNA adds skip connections to the first frame to handle occlusion (Ebert et al., 2017), SDC-Net samples past pixels with predicted displacements and kernels (Reda et al., 2018), and voxel flow resamples frames along predicted flow (Liu et al., 2017); SAVP and MCVD add stochastic and diffusion models (Lee et al., 2018; Voleti et al., 2022). In feature space, Deep Feature Flow propagates CNN features along optical flow instead of recomputing them (Zhu et al., 2017), F2F forecasts future convolutional features for semantic and instance segmentation (Luc et al., 2017; 2018), and F2M warps observed features with a forecast feature-motion field and blends them with a directly regressed forecast (Šarić et al., 2020). Recent robot world models reintroduce explicit motion: FlowDreamer predicts 3D scene flow before diffusion rendering (Guo et al., 2026a), AHEAD conditions token forecasts on optical-flow velocity (Syed et al., 2026), and object-centric models impose structure through slots (Wu et al., 2023; Mosbach et al., 2025); space-time memory networks copy past features forward by attention to propagate masks (Oh et al., 2019). CDNA, SNA and SDC-Net move *pixels* with action-conditioned or displaced kernels, and F2M and STM move features without actions. SHIFTWm carries this move-and-correct idea into latent world models: a learned soft local transport of frozen foundation-model patch features, conditioned on the action prefix, gated against the identity with a correction, decoded in parallel for every horizon, and usable as a plug-in head for published world models.



Figure 2: **SHIFTWM architecture.** Top: a predictor turns observed features and actions into one query per patch and step; SHIFTWM decodes all steps in parallel from observed features. Bottom: on a held-out window, each forecast is kept + moved + corrected observed features.

3 METHOD

Method: SHIFTWM. SHIFTWM is a latent world model whose output is a *motion* of the feature grids it has observed. For every future step and every patch, it predicts where in a local window of measured features the patch’s content comes from, whether the patch stays or moves, and a correction for content that motion cannot explain (Figure 2; the lower row shows each term of Equation (2) on a held-out DROID window).

3.1 PROBLEM SETTING

A frozen encoder E maps an image to a grid of $N = G \times G$ patch features $\mathbf{Z}_t \in \mathbb{R}^{N \times C}$. Given H observed grids $\mathbf{Z}_{-H+1:0}$, the actions between them $\mathbf{a}_{-H+1:-1}$, and a future action sequence $\mathbf{a}_{0:K-1}$, a world model predicts $\hat{\mathbf{Z}}_{1:K}$. The forecast $\hat{\mathbf{Z}}_k$ may depend only on $\mathbf{a}_{<k}$ (causality). Features are standardised per channel with train-split statistics that are shared across positions.

3.2 ARCHITECTURE

Memory encoder. Each observed patch feature is projected to width d and receives a spatial embedding, a frame embedding and an embedding of the action applied after that frame. L_e transformer layers over all $H \cdot N$ tokens give memory states $\mathbf{M} \in \mathbb{R}^{H \times N \times d}$. **Action-prefix queries.** A unidirectional GRU over embedded actions gives a causal prefix state, to which we add a horizon embedding to form $\mathbf{c}_k$. Query tokens start from the memory states of the last frame, and L_d decoder layers apply self-attention across patches and cross-attention to $\mathbf{M}$, each modulated by $\mathbf{c}_k$ through adaptive layer normalisation with zero-initialised gains (Peebles & Xie, 2023). The result is $\mathbf{h}_{k,i} \in \mathbb{R}^d$ for patch i at step k ; all K steps are decoded in parallel.

3.3 TRANSPORT AND GATED OUTPUT

For query patch i , let $\mathcal{W}(i)$ be the $w \times w$ spatial window around i in each of the last $S \leq H$ observed frames. Transport weights are

$$\pi_{k,i}(j) = \text{softmax}_{j \in \mathcal{W}(i)} \left(\frac{1}{\sqrt{d_k}} (\mathbf{W}_q \mathbf{h}_{k,i})^\top (\mathbf{W}_k \mathbf{M}_j) + \beta \mathbf{1}[j = (0, i)] \right), \quad (1)$$

where β is a learned identity bias toward the same location in the last frame. Transport moves *measured* features, not latent states: $\mathbf{T}_{k,i} = \sum_j \pi_{k,i}(j) \mathbf{Z}_j$. A per-patch gate $g_{k,i} = \sigma(\mathbf{w}_g^\top \mathbf{h}_{k,i} + b_g)$ and a correction $\mathbf{r}_{k,i} = \mathbf{s} \odot \mathbf{W}_o \mathbf{h}_{k,i}$ give

$$\hat{\mathbf{Z}}_{k,i} = (1 - g_{k,i}) \mathbf{Z}_{0,i} + g_{k,i} \mathbf{T}_{k,i} + \mathbf{r}_{k,i}. \quad (2)$$

$\mathbf{W}_o$ is zero-initialised and $b_g = -3$, so an untrained model starts close to persistence and learns departures from it. $\mathbf{s} \in \mathbb{R}^C$ is a learned per-channel scale (Table 3 compares it with a bounded tanh correction).

3.4 TRAINING

We minimise the mean squared error over all horizons, $\mathcal{L}_{\text{pred}} = \frac{1}{K} \sum_k \|\hat{\mathbf{Z}}_k - \mathbf{Z}_k\|^2$. Anchoring every forecast to observed features makes it easy to ignore actions: a forecast that stays close to $\mathbf{Z}_0$ scores well on average. For planning we therefore add an action-contrastive hinge that compares the error under the true future actions with the error under actions borrowed from another sample in the batch:

$$\mathcal{L}_{\text{act}} = \max(0, m - (\|\hat{\mathbf{Z}}(\tilde{\mathbf{a}}) - \mathbf{Z}\|^2 - \text{sg} \|\hat{\mathbf{Z}}(\mathbf{a}) - \mathbf{Z}\|^2)), \quad \mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{act}}. \quad (3)$$

All forecasting and plug-in results use $\lambda=0$, like every baseline; $\mathcal{L}_{\text{act}}$ is only a planning variant.

3.5 A MOTION MODEL THAT MOTIVATES TRANSPORT

We motivate Equation (2) with a simple generative model; Figure 3 shows the geometry for one patch. Suppose the grid at step k arises from motion plus innovation,

$$\mathbf{Z}_k = \mathbf{P}_k \mathbf{Z}_0 + \varepsilon_k, \quad (4)$$

where $\mathbf{P}_k \in \{0, 1\}^{N \times N}$ is a row-stochastic *motion* matrix (each patch copies one source patch $\pi(i)$ at integer-patch offsets; $\mathbf{P}_k = \mathbf{I}$ is a static scene; sub-patch motion is covered by soft weights between neighbouring sources) and ε_k is new content with $\mathbb{E}[\varepsilon_k | \mathbf{P}_k, \mathbf{Z}_0, \mathbf{a}] = \mathbf{0}$. Motion is uncertain given the context $c = (\mathbf{Z}_{\leq 0}, \mathbf{a}_{\leq k})$.

Proposition 1 (The optimal forecast is a transport). *Under Equation (4), the MSE-optimal forecast is $\mathbb{E}[\mathbf{Z}_k | c] = \bar{\mathbf{P}}_k \mathbf{Z}_0$ with $\bar{\mathbf{P}}_k = \mathbb{E}[\mathbf{P}_k | c]$, a row-stochastic matrix. If the support of each row of $\mathbf{P}_k$ lies in the transport window $\mathcal{W}$, then $\bar{\mathbf{P}}_k \mathbf{Z}_0$ can be approximated to arbitrary precision by the transport of Equation (1) with $g=1$ and $\mathbf{r}=\mathbf{0}$, given logits that the decoder can express.*

Proposition 1 characterises the optimum that *any* sufficiently expressive head could reach, Direct included: it shows that transport is an exact, well-conditioned parameterisation of that optimum under motion, not that transport alone is optimal; any advantage comes from this inductive bias, which we test empirically. Real features also change through viewpoint, lighting and deformation, which the correction $\mathbf{r}$ absorbs. An additive head $\mathbf{Z}_0 + \mathbf{W}_o \mathbf{h}$ instead has to write out the displacement in feature space:

Proposition 2 (Size of the regression target). *For a deterministic motion $\mathbf{P}$ with sources $\pi(i)$, the residual an additive head must output is $\mathbf{R} = (\mathbf{P} - \mathbf{I})\mathbf{Z}_0$ with $\|\mathbf{R}\|_F^2 = \sum_i \|\mathbf{z}_{0,\pi(i)} - \mathbf{z}_{0,i}\|^2$, the feature contrast along the motion. The residual required by SHIFTWm is $\mathbf{0}$ for every such $\mathbf{P}$ within the window.*

This concerns what each head must represent, not learnability: an additive head can learn to copy, but its target grows with motion and feature contrast. On static patches ($\mathbf{P} = \mathbf{I}$) the identity path is

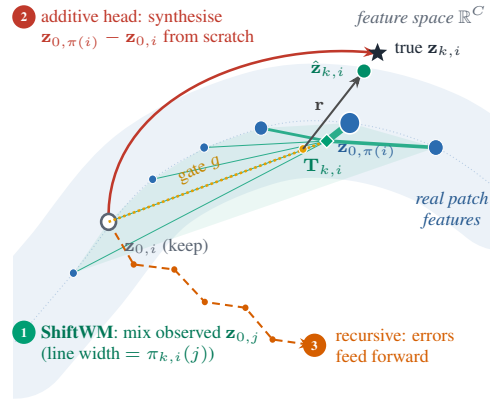


Figure 3: **One patch in feature space.** ① SHIFTWm mixes observed vectors (Prop. 1); ② additive heads synthesise the displacement (Prop. 2); ③ recursion feeds errors forward (Prop. 3).

exact, whereas a regression head must output exactly zero, and any error corrupts the static scene; Table 17 shows both effects. Finally, SHIFTWM, like the Direct baseline, decodes every horizon from measured features and never feeds its predictions back, so the next result contrasts recursive with direct decoding and does not bear on SHIFTWM versus Direct (proofs in Appendix B.2):

Proposition 3 (Recursive versus direct decoding). *Let a recursive predictor f be L -Lipschitz in its input with one-step error at most δ . Its k -step rollout error is bounded by $\delta \sum_{j=0}^{k-1} L^j$, which grows geometrically for $L > 1$. A direct predictor with per-horizon error δ_k incurs exactly δ_k , independent of its errors at other horizons.*

4 EXPERIMENTAL SETUP

We ask four questions. **(Q1)** Does transport improve held-out forecasting over matched predictors on real robot and surgical video? **(Q2)** Does the SHIFTWM head improve *published* world models? **(Q3)** Why does it work? **(Q4)** Which components matter, and what happens in closed-loop planning? Selection uses validation splits only; test splits are scored once.

Datasets (Figure 4, Tables 5 and 6). **DROID** (Khazatsky et al., 2024): real Franka episodes split by recording session; we train on the primary exterior camera and test transfer to the second. One step is 0.33 s with a 35-D Cartesian and gripper block; we forecast $K=10$ steps (3.3 s). **Open-H Hamlyn dVRK** (Open-H Embodiment Consortium, 2026): seven surgical training tasks with 80-D bimanual action blocks, episode-disjoint splits per task. **IWS** (Wang et al., 2026b; Zhang et al., 2026a): real bimanual PushT, Box and Rope, scored on the official RLA-WM validation handles. **BridgeData V2** (Walke et al., 2023), **RT-1** (Open X-Embodiment Collaboration, 2024) and **Language-Table** (Lynch et al., 2023): pinned Open-X subsets. **Planning**: PushT, TwoRoom and Reacher with the official LeWM protocol (Maes et al., 2026b;a), and PushT and Wall with the official DINO-WM protocol.

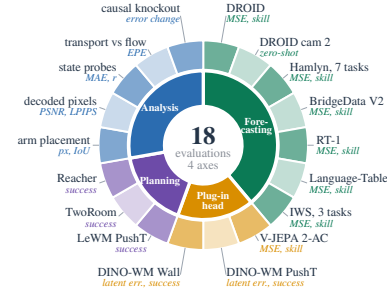


Figure 4: **Evaluation wheel**: 18 evaluations across four axes (inner), with their metrics (outer).

Baselines. All learned predictors share a frozen DINOv2-S/14 encoder (16×16 patches, no pooling), the same memory encoder and decoder (width 384, $L_e=L_d=4$, ≈ 22 M parameters), optimiser, schedule, steps and batch size (Appendix B.1); they differ only in the output head and rollout. **Direct** replaces Equation (2) with $\mathbf{Z}_0 + \mathbf{W}_o \mathbf{h}_k$; it has the same cross-patch attention, so it controls for spatial communication. **AR** predicts a residual to the most recent grid and is rolled out recursively, with gradients through the rollout. **AR-TF** follows DINO-WM: it is trained with teacher forcing to predict absolute features and is rolled out at test time. **Persistence** copies $\mathbf{Z}_0$, and **Linear** extrapolates the last feature difference. As published models we use V-JEPA 2-AC and DINO-WM, with and without the SHIFTWM head (Section 5.2).

Metrics and protocol. The primary metric is standardised feature MSE on held-out episodes, averaged over horizons and at the final step. We also report *skill*, the fraction of persistence error a model removes; being scale-free, it compares models in different feature spaces. Episodes are the statistical unit (Section 5.3). Planning uses CEM with the feature distance to the goal grid as cost.

5 RESULTS AND DISCUSSION

5.1 HELD-OUT FORECASTING ACROSS ROBOTS AND SURGERY (Q1)

Table 1 compares SHIFTWM with predictors that share its backbone and differ only in how the forecast is formed. On held-out DROID, SHIFTWM has the lowest error of all methods: **7.6%** lower than Direct, **12.2%** lower than the rollout-trained AR and **34.3%** lower than the DINO-WM-style AR-TF, whose teacher-forced absolute predictions are worse than persistence once rolled out;

Table 1: **Held-out forecasting error** (feature MSE ↓, mean over training seeds; DROID: 3). **Best, second**; †: paired 95% CI vs. best baseline excludes 0. [Interim: seeds completed – Hamlyn: ar 1/3, ar-tf 1/3, direct 1/3, shiftwm 1/3.]

Method	DROID		DROID cam 2	Open-H	IWS	Bridge	RT-1	Lang.
	avg	$k=10$	(zero-shot)	surgical		V2	(Fractal)	Table
Persistence	0.249	0.326	pending	0.444	pending	pending	pending	0.365
Linear extrapolation	4.812	11.855	pending	10.824	pending	pending	pending	8.437
AR-TF (DINO-WM-style)	0.275	0.417	pending	0.285	pending	pending	pending	pending
AR (rollout-trained)	0.206	0.264	pending	0.235	pending	pending	pending	0.122
Direct (cross-attn)	0.196	0.250	pending	0.234	pending	pending	pending	0.120
SHIFTWM (ours)	0.181†	0.231†	pending	0.226†	pending	pending	pending	0.111†
error reduction vs. best baseline	+7.6%	+7.3%	pending	+3.6%	pending	pending	pending	+6.9%

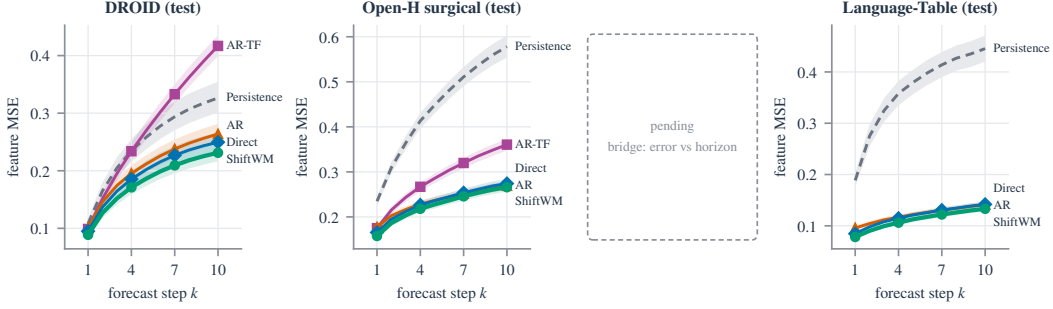


Figure 5: **Forecast error vs. horizon on held-out episodes.** Bands: 95% bootstrap intervals over episodes.

it removes 27.5% of the persistence error, against 21.6% for Direct and 17.4% for AR. On the seven surgical tasks of Open-H the ordering is the same but the margins are small: 3.6% below Direct and 4.0% below AR, currently from 1 training seed(s) per method. On Language-Table, where a robot pushes rigid blocks, SHIFTWM has **6.9%** lower error than Direct and **8.6%** lower than AR (1 seed). Paired 95% intervals of the gain over the best baseline are [6.8, 8.3]% on DROID (resampling 57 test sessions) and [6.4, 7.4]% on Language-Table. Recursive predictors accumulate error as they consume their own forecasts (Figure 5; Proposition 3), while SHIFTWM is lowest at every horizon and on all seven surgical tasks (Tables 10 and 11). It also ranks actions best (true vs. another episode’s actions): **92.0%** of DROID windows, against 89.8% (Direct) and 81.0% (AR).

5.2 SHIFTWM AS A PLUG-IN FOR PUBLISHED WORLD MODELS (Q2)

Because SHIFTWM is an output head, we attach it to two published models without other changes: it blends the backbone’s prediction $\mathbf{P}$ with transported observed features, $\hat{\mathbf{Z}} = (1 - g)\mathbf{P} + g\mathbf{T}$, with the gate initialised near zero so training starts from the unmodified model. Each pair shares data, schedule, loss and seed. **V-JEPA 2-AC**. The released model (Assran et al., 2025) (ViT-g encoder, 300M predictor, trained on DROID) removes 4.1% of persistence error zero-shot on our test windows, 24.6% after fine-tuning, and **36.4%** after fine-tuning *with* the head under the identical budget (Table 2): **15.7%** lower error and 16.9% lower on moving patches for 0.16M added parameters. The gate opens from 1.8% to 56% at the selected checkpoint and 65% by the end of fine-tuning, and validation error is lower at every evaluation for both seeds (Figure 9). V-JEPA 2-AC’s training data likely include our test trajectories, which mainly affects the zero-shot number. **DINO-WM**. With the official code (Zhou et al., 2025), the head lowers latent error by **8.0%** on PushT but *raises* it by 18.4% on Wall, where DINO-WM conditions on a single history frame ($S=1$): after the first rollout step the head’s “observed” token is DINO-WM’s own prediction, which we hypothesise (unproven) contributes to the regression. Planning success changes from 62.0% to *pending*% (PushT) and *pending*% to *pending*% (Wall); budgets are reduced and identical for both arms (Appendix B.3).

Table 2: **SHIFTWM as a plug-in head for published world models.** Same data, schedule and loss for each pair; only the head differs. Top: V-JEPA 2-AC on DROID test (its own feature space). Bottom: DINO-WM, official code and protocol.

V-JEPA 2-AC (ViT-g, 1.3B) on DROID		MSE ↓	moving ↓	static ↓	skill (%) ↑
V-JEPA 2-AC, zero-shot		0.504	0.815	0.400	4.1
V-JEPA 2-AC, fine-tuned (2 seeds)		0.396	0.741	0.281	24.6
V-JEPA 2-AC, fine-tuned + SHIFTWM head (2 seeds)		0.334	0.616	0.240	36.4

Env	DINO-WM (ViT-S/14)	latent err. ↓	SSIM ↑	LPIPS ↓	success (%) ↑
PushT	DINO-WM	0.105	0.976	0.014	62.0
PushT	DINO-WM + SHIFTWM head	0.097	0.977	0.013	pending
Wall	DINO-WM	0.026	0.993	0.009	pending
Wall	DINO-WM + SHIFTWM head	0.031	0.988	0.026	pending

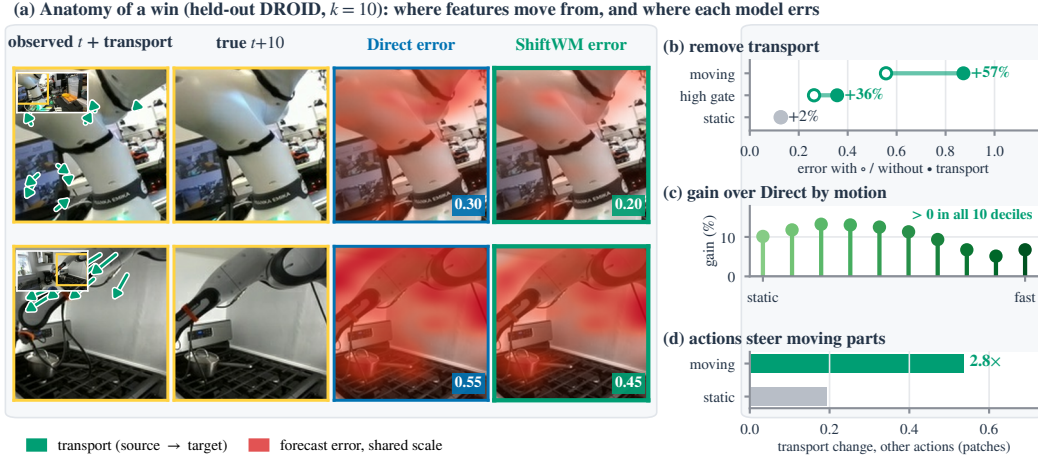


Figure 6: **Where and why SHIFTWM wins (held-out DROID, $k=10$).** (a) Transport arrows, true future, and Direct vs. SHIFTWM error on the same zoom. (b) Removing transport. (c) Gain per motion decile. (d) Action steering.

5.3 WHY TRANSPORT HELPS (Q3)

Where do the gains come from? This analysis tests the target-size argument (Proposition 2). We split every test window into *moving* patches (the 25% with the largest true feature change, a property of the data shared by all methods) and *static* patches (Table 17). On moving patches, transport predicts where content goes: SHIFTWM has **7.2%** lower error than Direct and **10.0%** lower than AR. On static patches, every other learned predictor is *worse than copying*, since regenerating the grid corrupts background that should not change; SHIFTWM keeps it nearly intact, and it is the only learned model below persistence (**2.1%** lower; 15.5% lower than AR). Across all ten deciles of true motion, SHIFTWM has lower error than Direct (**+5.1% to +13.2%**), with the gains concentrated on the arm and gripper (Figure 6).

Reliance on transport. Setting the gate to zero at test time, so that each forecast is $\mathbf{Z}_0 + \mathbf{r}$, raises the error by 56.8% on moving patches and by 35.9% on patches with a high learned gate, but only by 1.8% on static patches: the trained model relies on transport, not on the correction, for moving content. This test-time intervention shows reliance, not necessity; the controlled evidence is the retrained Direct head without transport, over which SHIFTWM lowers moving-patch error by 7.2%. Replacing the future actions with those of another episode changes the transport field **2.8×** more on moving than on static patches, so the actions steer *where* features are moved.

Is the future a transport? As a model-free check of Proposition 1, on moving patches of 3,923 DROID test windows the single observed feature in the transport window closest to the true future (an *oracle move*) cuts the error of staying put by 51% at $k=10$: half of the future is already in the observed grid, displaced. SHIFTWM recovers **79%** of this gain (Direct 71%, AR 69%), is better

Table 3: **Ablations on DROID validation.** Each row changes one component of SHIFTWM.

	Variant	Avg	K	Rank acc.
	SHIFTWM (window 7, 3 sources, scaled correction)	pending	pending	pending
Transport	no transport (= Direct)	pending	pending	pending
	global window	pending	pending	pending
Memory	window $w = 3 / w = 11$	pending	pending	pending
	last frame only ($S=1$)	pending	pending	pending
Correction	tanh-bounded	pending	pending	pending
	none (pure transport)	pending	pending	pending
Actions	action-free	pending	pending	pending
	+ action-contrastive ($\lambda=1$)	pending	pending	pending
Encoder	DINOv2-B/14	pending	pending	pending
	DINOv3-S/16	pending	pending	pending
	DINOv2-S, 4×4 pooled grid	pending	pending	pending

Table 4: **Planning success (%)** at the official protocol (50 episodes $\times$ 3 seeds). LeWM (end-to-end encoder) is shown for reference.

Predictor	PushT	TwoRoom	Reacher	s / plan (PushT)
LeWM (released) (Maes et al., 2026b)	88.0	85.3	84.7	0.41
AR-TF (DINO-WM-style)	pending	pending	pending	pending
AR (rollout-trained)	pending	pending	pending	pending
Direct	pending	pending	pending	pending
SHIFTWM	pending	pending	pending	pending
SHIFTWM + action-contrastive	pending	pending	pending	pending

than Direct on 64% of moving patches, and has lower error than Direct and AR in all 100 horizon-by-motion cells (Figures 12 and 13). Against SAM 2.1 masks of the true future (Appendix E.2), SHIFTWM places the DROID arm +2.1 px closer than AR but only marginally closer than Direct (+0.6 px, CI [+0.0, +1.1]).

Statistical analysis. All comparisons are paired: we bootstrap per-episode differences (recording sessions for DROID; 10,000 resamples, 95% intervals), Holm-adjusted (Appendix C.6). Seeds per method: DROID 3, Hamlyn 1, Language-Table 1, V-JEPA 2-AC 2; with one seed, intervals reflect episode variance only.

5.4 WHAT MATTERS (Q4)

Table 3 removes or alters one component at a time on DROID. pending

5.5 CLOSED-LOOP PLANNING (Q4)

All predictors use the same CEM planner with the official LeWM budget (Table 9). Our harness reproduces the released LeWM checkpoint at 88% on PushT (the LeWM paper reports 96 ± 2.8 over training seeds; Appendix C.5). pending

6 CONCLUSION

SHIFTWM forecasts by moving what it has already seen, plus a small correction. Against matched predictors it has the lowest held-out error on real robot video and, by a small margin, on surgical robot tasks; as a plug-in head it improves V-JEPA 2-AC and DINO-WM on PushT, and it makes predicted motion explicit. The analyses trace the gain mainly to transport on moving content.

Limitations. The plug-in does not help uniformly: on DINO-WM Wall, with a single history frame, it raises latent error. Gains on the surgical tasks are small, and arm placement improves only marginally over Direct. Language-Table is split by segment, so test and training segments may share a play session. Newly revealed content depends on the correction, the forecast is deterministic, pixel metrics rely on a separate decoder, and the surgical data are scene-camera views of dVRK training tasks, not in-vivo endoscopy.

REPRODUCIBILITY STATEMENT

Code, configurations, per-episode evaluation records and trained checkpoints will be released. Appendix B.1 lists every hyperparameter. Dataset splits are deterministic and documented in Appendix C.2. All datasets are public: DROID (CC-BY-4.0), Open-H-Embodiment (CC-BY-4.0), the RLA-WM/IWS release, and the LeWM planning datasets (MIT).

ETHICS STATEMENT

The surgical data are public recordings of robotic procedures on phantoms and ex-vivo tissue, released under CC-BY-4.0. We make no clinical claims, and the models are not intended for clinical use.

AI USE STATEMENT

In this work, we used generative AI tools to assist with writing and debugging research code, orchestrating experiments, surveying related work, and editing the manuscript text. All AI-assisted code was tested, and every reported number is produced by logged runs that the authors reviewed. Bibliographic entries were verified against the original sources. We take responsibility for the final content of this work, including text, claims and artifacts produced with the aid of generative AI.

REFERENCES

- Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2405.12399>.
- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, et al. V-JEPA 2: Self-Supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. URL <https://arxiv.org/abs/2506.09985>.
- Federico Baldassarre, Marc Szafraniec, Basile Terver, Vasil Khalidov, Francisco Massa, Yann LeCun, Patrick Labatut, Maximilian Seitzer, and Piotr Bojanowski. Back to the features: DINO as a foundation for video world models. *arXiv preprint arXiv:2507.19468*, 2025. URL <https://arxiv.org/abs/2507.19468>.
- Randall Balestrierio and Yann LeCun. LeJEPa: Provable and scalable Self-Supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025. URL <https://arxiv.org/abs/2511.08544>.
- Amir Bar, Gaoyue Zhou, Danny Tran, Trevor Darrell, and Yann LeCun. Navigation world models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. URL <https://arxiv.org/abs/2412.03572>.
- Gabrijel Boduljak, Yushi Lan, Christian Rupprecht, and Andrea Vedaldi. VFMP: World modeling by forecasting vision foundation model features. *arXiv preprint arXiv:2512.11225*, 2025. URL <https://arxiv.org/abs/2512.11225>.
- Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, 2024. URL <https://arxiv.org/abs/2402.15391>.
- Bert De Brabandere, Xu Jia, Tinne Tuytelaars, and Luc Van Gool. Dynamic filter networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. URL <https://arxiv.org/abs/1605.09673>.
- Haoyi Duan, Hong-Xing Yu, Sirui Chen, Li Fei-Fei, and Jiajun Wu. WorldScore: A unified evaluation benchmark for world generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. URL <https://arxiv.org/abs/2504.00983>.

-
- Frederik Ebert, Chelsea Finn, Alex X. Lee, and Sergey Levine. Self-supervised visual planning with temporal skip connections. In *Conference on Robot Learning (CoRL)*, volume 78 of *PMLR*, 2017. URL <https://arxiv.org/abs/1710.05268>.
- Chelsea Finn, Ian Goodfellow, and Sergey Levine. Unsupervised learning for physical interaction through video prediction. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL <https://arxiv.org/abs/1605.07157>.
- Yixin Gao, S. Swaroop Vedula, Carol E. Reiley, Narges Ahmidi, Balakrishnan Varadarajan, Henry C. Lin, Lingling Tao, Luca Zappella, Benjamín Béjar, David D. Yuh, Chi Chiung Grace Chen, René Vidal, Sanjeev Khudanpur, and Gregory D. Hager. JHU-ISI gesture and skill assessment working set (JIGSAWS): A surgical activity dataset for human motion modeling. In *Modeling and Monitoring of Computer Assisted Interventions (M2CAI), MICCAI Workshop*, 2014. URL https://cirl.lcsr.jhu.edu/research/hmm/datasets/jigsaws_release/.
- Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *IEEE Robotics and Automation Letters*, 2026a. doi: 10.1109/LRA.2026.3653273. URL <https://arxiv.org/abs/2505.10075>.
- Yanjiang Guo, Lucy Xiaoyang Shi, Jianyu Chen, and Chelsea Finn. Ctrl-World: A controllable generative world model for robot manipulation. In *International Conference on Learning Representations (ICLR)*, 2026b. URL <https://arxiv.org/abs/2510.10125>.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, 2019. URL <https://arxiv.org/abs/1811.04551>.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025. doi: 10.1038/s41586-025-08744-2. URL <https://arxiv.org/abs/2301.04104>.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.16828>.
- Jesse Haworth, Jue-Tung Chen, Nigel Nelson, Ji Woong Kim, Masoud Moghani, Chelsea Finn, and Axel Krieger. SutureBot: A precision framework & benchmark for autonomous End-to-End suturing. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. URL <https://arxiv.org/abs/2510.20965>.
- Yufan He, Pengfei Guo, Mengya Xu, Zhaoshuo Li, Andriy Myronenko, Dillan Imans, Bingjie Liu, Dongren Yang, Mingxue Gu, Yongnan Ji, et al. Cosmos-H-Surgical: Learning surgical robot policies from videos via world modeling. *arXiv preprint arXiv:2512.23162*, 2025. URL <https://arxiv.org/abs/2512.23162>.
- Siqiao Huang, Partha Kaushik, Michael Chen, Hengkai Pan, Kaiwen Geng, Omar Chehab, Fernando Moreno-Pino, and Max Simchowitz. Nano world models: A minimalist implementation of future video prediction. *arXiv preprint arXiv:2605.23993*, 2026. URL <https://arxiv.org/abs/2605.23993>.
- Efstathios Karypidis, Ioannis Kakogeorgiou, Spyros Gidaris, and Nikos Komodakis. DINO-Foresight: Looking into the future with DINO. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2412.11673>.
- Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems (RSS)*, 2024. URL <https://arxiv.org/abs/2403.12945>.

-
- Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos policy: Fine-Tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026. URL <https://arxiv.org/abs/2601.16163>.
- Saurabh Koju, Saurav Bastola, Prashant Shrestha, Sanskar Amgain, Yash Raj Shrestha, Rudra P. K. Poudel, and Binod Bhattarai. Surgical vision world model. In *Data Engineering in Medical Imaging (DEMI), MICCAI Workshops*, Lecture Notes in Computer Science, 2025. doi: 10.1007/978-3-032-08009-7_1. URL <https://arxiv.org/abs/2503.02904>.
- Alex X. Lee, Richard Zhang, Frederik Ebert, Pieter Abbeel, Chelsea Finn, and Sergey Levine. Stochastic adversarial video prediction. *arXiv preprint arXiv:1804.01523*, 2018. URL <https://arxiv.org/abs/1804.01523>.
- Chenxin Li, Hengyu Liu, Yifan Liu, Brandon Y. Feng, Wuyang Li, Xinyu Liu, Zhen Chen, Jing Shao, and Yixuan Yuan. Endora: Video generation models as endoscopy simulators. In *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, volume 15006 of *Lecture Notes in Computer Science*, pp. 230–240, 2024. doi: 10.1007/978-3-031-72089-5_22. URL <https://arxiv.org/abs/2403.11050>.
- Dacheng Li, Yunhao Fang, Yukang Chen, Shuo Yang, Shiyi Cao, Justin Wong, Michael Luo, Xiaolong Wang, Hongxu Yin, Joseph E. Gonzalez, et al. WorldModelBench: Judging video generation models as world models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. URL <https://arxiv.org/abs/2502.20694>.
- Huanrong Liu, Weiliang Huang, Bob Zhang, Weichao Cai, Chunlin Tian, and Qingbiao Li. Surg-WMBench: A Vision-Based benchmark for World-Modeling surgical instrument motion planning. *arXiv preprint arXiv:2608.08070*, 2026. URL <https://arxiv.org/abs/2608.08070>.
- Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision (ECCV)*, 2024.
- Ziwei Liu, Raymond A. Yeh, Xiaoou Tang, Yiming Liu, and Aseem Agarwala. Video frame synthesis using deep voxel flow. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017. URL <https://arxiv.org/abs/1702.02463>.
- Po-Chien Luan, Zimin Xia, Wuyang Li, Yang Gao, and Alexandre Alahi. Drift-Resistant navigation world model with anchored epipolar guidance. *arXiv preprint arXiv:2605.24761*, 2026. URL <https://arxiv.org/abs/2605.24761>.
- Pauline Luc, Natalia Neverova, Camille Couprie, Jakob Verbeek, and Yann LeCun. Predicting deeper into the future of semantic segmentation. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.
- Pauline Luc, Camille Couprie, Yann LeCun, and Jakob Verbeek. Predicting future instance segmentation by forecasting convolutional features. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.
- Corey Lynch, Ayzaan Wahid, Jonathan Tompson, Tianli Ding, James Betker, Robert Baruch, Travis Armstrong, and Pete Florence. Interactive language: Talking to robots in real time. *IEEE Robotics and Automation Letters*, 2023. URL <https://arxiv.org/abs/2210.06407>.
- Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation. *arXiv preprint arXiv:2605.21800*, 2026a. URL <https://arxiv.org/abs/2605.21800>.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026b. URL <https://arxiv.org/abs/2603.19312>.

-
- Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are Sample-Efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2209.00588>.
- Malte Mosbach, Jan Niklas Ewertz, Angel Villar-Corrales, and Sven Behnke. SOLD: Slot Object-Centric latent dynamics models for relational manipulation learning from pixels. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025. URL <https://arxiv.org/abs/2410.08822>.
- Saman Motamed, Laura Culp, Kevin Swersky, Priyank Jaini, and Robert Geirhos. Do generative video models understand physical principles? In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2026. doi: 10.1109/WACV61042.2026.00099. URL <https://arxiv.org/abs/2501.09038>.
- Nilaksh, Saurav Jha, Artem Zholus, and Sarath Chandar. Reconstruction or semantics? what makes a latent space useful for robotic world models. *arXiv preprint arXiv:2605.06388*, 2026. URL <https://arxiv.org/abs/2605.06388>.
- NVIDIA. Cosmos world foundation model platform for physical AI. *arXiv preprint arXiv:2501.03575*, 2025a. URL <https://arxiv.org/abs/2501.03575>.
- NVIDIA. World simulation with video foundation models for physical AI. *arXiv preprint arXiv:2511.00062*, 2025b. URL <https://arxiv.org/abs/2511.00062>.
- Chinedu Innocent Nwoye, Tong Yu, Cristians Gonzalez, Barbara Seeliger, Pietro Mascagni, Didier Mutter, Jacques Marescaux, and Nicolas Padoy. Rendezvous: Attention mechanisms for the recognition of surgical action triplets in endoscopic videos. *Medical Image Analysis*, 78:102433, 2022. URL <https://arxiv.org/abs/2109.03223>.
- Seoung Wug Oh, Joon-Young Lee, Ning Xu, and Seon Joo Kim. Video object segmentation using space-time memory networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9226–9235, 2019.
- Open-H-Embodiment Consortium. Open-H-Embodiment: A Large-Scale dataset for enabling foundation models in medical robotics. *arXiv preprint arXiv:2604.21017*, 2026. URL <https://arxiv.org/abs/2604.21017>.
- Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. doi: 10.1109/ICRA57147.2024.10611477. URL <https://arxiv.org/abs/2310.08864>.
- Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. URL <https://arxiv.org/abs/2304.07193>.
- Wentao Pan, Wuyang Li, Shengyuan Liu, Xinyu Liu, Hengyu Liu, and Yixuan Yuan. SurgVista: Long-Horizon surgical world modeling with plausible Instrument-Tissue dynamics. *arXiv preprint arXiv:2606.19889*, 2026. URL <https://arxiv.org/abs/2606.19889>.
- Jack Parker-Holder and Shlomi Fruchter. Genie 3: A new frontier for world models. Google DeepMind blog, August 2025. URL <https://deepmind.google/discover/blog/genie-3-a-new-frontier-for-world-models/>.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4195–4205, 2023.
- Francois Porcher, Nicolas Carion, Karteek Alahari, and Shizhe Chen. Flow matching in feature space for stochastic world modeling. *arXiv preprint arXiv:2606.29059*, 2026. URL <https://arxiv.org/abs/2606.29059>.

- Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations (ICLR)*, 2025.
- Fitsum A. Reda, Guilin Liu, Kevin J. Shih, Robert Kirby, Jon Barker, David Tarjan, Andrew Tao, and Bryan Catanzaro. SDC-Net: Video prediction using spatially-displaced convolution. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 718–733, 2018.
- Josip Šarić, Sacha Vražić, and Siniša Šegvić. Warp to the future: Joint forecasting of features and feature motion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. URL <https://arxiv.org/abs/2508.10104>.
- Ssharvien Kumar Sivakumar, Akwele Johnson, Anirudh Dhingra, Yannik Frisch, Ghazal Ghazaei, and Anirban Mukhopadhyay. SWoMo: Neuro-symbolic world model for cataract surgery simulation. *arXiv preprint arXiv:2605.16530*, 2026. URL <https://arxiv.org/abs/2605.16530>.
- Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from Reward-Free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2502.14819>.
- Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-Space predictive world model for dynamic VLA manipulation. *arXiv preprint arXiv:2606.02486*, 2026. URL <https://arxiv.org/abs/2606.02486>.
- Zachary Teed and Jia Deng. RAFT: Recurrent All-Pairs field transforms for optical flow. In *European Conference on Computer Vision (ECCV)*, 2020. doi: 10.1007/978-3-030-58536-5_24. URL <https://arxiv.org/abs/2003.12039>.
- Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with Joint-Embedding predictive world models? *Transactions on Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2512.24497>.
- Andru P. Twinanda, Sherif Shehata, Didier Mutter, Jacques Marescaux, Michel de Mathelin, and Nicolas Padoy. EndoNet: A deep architecture for recognition tasks on laparoscopic videos. *IEEE Transactions on Medical Imaging*, 36(1):86–97, 2017. doi: 10.1109/TMI.2016.2593957. URL <https://arxiv.org/abs/1602.03012>.
- Arun Venkatraman, Martial Hebert, and J. Andrew Bagnell. Improving multi-step prediction of learned time series models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.
- Vikram Voleti, Alexia Jolicoeur-Martineau, and Christopher Pal. MCVD: Masked conditional video diffusion for prediction, generation, and interpolation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2205.09853>.
- Carl Vondrick and Antonio Torralba. Generating the future with adversarial transformers. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. URL https://openaccess.thecvf.com/content_cvpr_2017/html/Vondrick_Generating_the_Future_CVPR_2017_paper.html.
- Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, et al. BridgeData V2: A dataset for robot learning at scale. In *Conference on Robot Learning (CoRL)*, 2023. URL <https://arxiv.org/abs/2308.12952>.

-
- 756 Jiawei Wang, Ke Rui, Yushen Zuo, Yichun Feng, and Minglei Li. Decision-metric alignment in
757 latent world models: Diagnostics and action-conditioned objectives for mpc planning. *arXiv*
758 *preprint arXiv:2608.18746*, 2026a. URL <https://arxiv.org/abs/2608.18746>.
759
- 760 Yixuan Wang, Rhythm Syed, Fangyu Wu, Mengchao Zhang, Aykut Onol, Jose Barreiros, Hooshang
761 Nayyeri, Tony Dear, Huan Zhang, and Yunzhu Li. Interactive world simulator for robot policy
762 training and evaluation. *arXiv preprint arXiv:2603.08546*, 2026b. URL <https://arxiv.org/abs/2603.08546>.
763
- 764 Ziyi Wu, Nikita Dvornik, Klaus Greff, Thomas Kipf, and Animesh Garg. SlotFormer: Unsupervised
765 visual dynamics simulation with Object-Centric models. In *International Conference on Learning*
766 *Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.05861>.
767
- 768 Sherry Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Leslie Kaelbling, Dale Schuur-
769 mans, and Pieter Abbeel. Learning interactive Real-World simulators. In *International Confer-*
770 *ence on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06114>.
771
- 772 Jewon Yeom, Hanseul Kim, Jeongjae Park, Sungmok Jung, Jaejin Lee, and Taesup Kim. What
773 makes video world model latents Action-Relevant: Prediction over reconstruction. *arXiv preprint*
774 *arXiv:2606.07687*, 2026. URL <https://arxiv.org/abs/2606.07687>.
775
- 776 Shicheng Yin, Kaixuan Yin, Weixing Chen, Yang Liu, Guanbin Li, and Liang Lin. DDP-WM:
777 Disentangled dynamics prediction for efficient world models. *arXiv preprint arXiv:2602.01780*,
778 2026. URL <https://arxiv.org/abs/2602.01780>.
779
- 780 Hu Yue, Siyuan Huang, Yue Liao, Shengcong Chen, Pengfei Zhou, Liliang Chen, Maoqing Yao, and
781 Guanghui Ren. EWM Bench: Evaluating scene, motion, and semantic quality in embodied world
782 models. *arXiv preprint arXiv:2505.09694*, 2025. URL <https://arxiv.org/abs/2505.09694>.
783
- 784 Xinyu Zhang, Zhengtong Xu, Yutian Tao, Yeping Wang, Yu She, and Abdeslam Boularias. Learning
785 visual feature-based world models via residual latent action. *arXiv preprint arXiv:2605.07079*,
786 2026a. URL <https://arxiv.org/abs/2605.07079>.
787
- 788 Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong,
789 Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, et al. Delta-JEPA: Learning Action-
790 Sensitive world models via latent difference decoding. *arXiv preprint arXiv:2606.31232*, 2026b.
791 URL <https://arxiv.org/abs/2606.31232>.
792
- 793 Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-
794 trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Con-*
795 *ference on Machine Learning (ICML)*, volume 267 of *PMLR*, pp. 79115–79135, 2025. URL
796 <https://arxiv.org/abs/2411.04983>.
797
- 798 Fangqi Zhu, Hongtao Wu, Song Guo, Yuxiao Liu, Chilam Cheang, and Tao Kong. IRASim: A Fine-
799 Grained world model for robot manipulation. In *Proceedings of the IEEE/CVF International Con-*
800 *ference on Computer Vision (ICCV)*, 2025. URL <https://arxiv.org/abs/2406.14540>.
801
- 802 Xizhou Zhu, Yuwen Xiong, Jifeng Dai, Lu Yuan, and Yichen Wei. Deep feature flow for video
803 recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*
804 *(CVPR)*, 2017.
805
806
807
808
809

The appendix follows the order of the main paper.

Appendix	Contents
A Background and problem setting	the task and why it matters, why existing world models fall short, notation, related formulations, extended related work
B Extended method	architecture, proofs, plug-in heads
C Extended experimental details	benchmarks, datasets, baselines, training and compute, planning protocol, statistics
D Extended results	per-horizon and per-task results, ablations and recipes, plug-in dynamics, decoded pixels, probes, planning, efficiency
E Extended analysis and discussion	mechanism analyses, segmentation view, qualitative rollouts, failure cases, discussion

A BACKGROUND AND PROBLEM SETTING

A.1 THE TASK AND WHY IT MATTERS

Task. An action-conditioned latent world model receives the last H observed frames of a scene, encoded by a frozen visual foundation model into grids of patch features, the actions applied between them, and a candidate sequence of K future actions. It must predict the patch-feature grids of the next K frames, without seeing them. It never generates pixels: the forecast lives in the same feature space that downstream consumers read.

Why it is useful. Such forecasts serve three uses. (i) *Planning*: a model-predictive planner samples many candidate action sequences, forecasts each, and executes the one whose forecast is closest to a goal (Zhou et al., 2025; Maes et al., 2026b); forecast accuracy and sensitivity to the actions decide which candidate wins. (ii) *Control and policy evaluation*: a world model can score a policy’s proposed actions before they are executed on a real robot. (iii) *Anticipation*: in surgical robotics, forecasting where instruments and tissue will be a few seconds ahead is a building block for assistance and safety monitoring. All three need forecasts that are accurate many steps ahead, respond to the actions, and hold on scenes not seen in training.



Figure 7: **The forecasting problem on real held-out episodes.** Three observed frames and the future actions are given; features at future steps are predicted. Right: true feature change over the 16×16 grid.

An example. Figure 7 shows the task on two held-out episodes, one from DROID (a Franka arm in a real kitchen) and one from the Open-H Hamlyn surgical tasks (a dVRK instrument over a training phantom). The world model receives the last $H=3$ frames $t-2, t-1, t$, encoded by a frozen DINOv2 encoder into 16×16 grids of 384-D patch features, the actions applied between them, and a sequence of $K=10$ future action blocks (the commanded end-effector trajectory, bottom rows). It must predict the feature grids $\mathbf{Z}_{t+1}, \dots, \mathbf{Z}_{t+10}$, 3.3 s into the future, without seeing any future frame. The episodes are chosen by a fixed rule (largest feature change among test episodes; Appendix E.3).

The rightmost panels show where the features actually change between t and $t+10$. Change is concentrated on the robot and on the objects it touches: in these two windows a quarter of the patches carry more than 40% of the total change, while large parts of the scene change little. This is the structure that SHIFTWm exploits: most future patch features are already present in the observed grid, displaced by motion, and only a small part is new content.

A.2 WHY EXISTING WORLD MODELS FALL SHORT

Current latent world models regress the complete feature grid at every future step, and most roll out recursively (Section 2). Our matched baselines isolate these two design choices (Section 4), and the held-out results show three concrete failure modes.

1. **Regenerating static content corrupts it.** On static DROID patches, every learned regression predictor (Direct, AR, AR-TF) has higher error than simply copying the last observed frame (Table 17). A regression head must output exactly zero change there, and any approximation error damages background that should not change.
2. **Regenerating moving content blurs it.** On moving patches, an additive head must write out the feature contrast along the motion (Proposition 2). At $k=10$, Direct and AR retain less of the true spatial contrast than SHIFTWm (Figure 11b), and SHIFTWm has 7.2% and 10.0% lower moving-patch error than Direct and AR (Table 17).
3. **Recursion compounds error.** Predictors that consume their own forecasts accumulate error with the horizon (Proposition 3, Figure 5). The DINO-WM-style AR-TF, trained with teacher forcing, is worse than persistence on DROID once rolled out, and SHIFTWm has 12.2% lower error than the rollout-trained AR (Table 1).

At the same time, much of the future is already observed: on moving DROID patches, copying the single best observed feature from the transport window removes 51% of the error of staying put (Figure 12b). SHIFTWm is designed around this structure: it keeps static content through an exact identity path, moves observed content through transport, and decodes every horizon from measured features.

A.3 NOTATION

Symbol	Meaning
$\mathbf{Z}_t \in \mathbb{R}^{N \times C}$	frozen patch features of frame t ($N=256$ patches, $C=384$ for DINOv2-S)
H, K	number of observed frames (3) and forecast horizon (10 steps of 1/3 s)
$\mathbf{a}_{0:K-1}$	future action blocks (commanded poses and gripper states)
$\mathbf{h}_{k,i}$	query state of patch i at step k after the action-conditioned decoder
$\mathcal{W}(i), w, S$	transport window around patch i , its width (7) and number of source frames (3)
$\pi_{k,i}(j)$	transport weight from observed source patch j to target patch i at step k
$\mathbf{T}_k, g_k, \mathbf{r}_k$	transported features, gate and correction; $\hat{\mathbf{Z}}_k = (1 - g_k)\mathbf{Z}_0 + g_k\mathbf{T}_k + \mathbf{r}_k$
$\mathbf{P}_k, \epsilon_k$	motion matrix and new content in the generative model of Section 3.5
skill	$1 - \text{MSE}/\text{MSE}_{\text{persistence}}$, the fraction of persistence error removed

A.4 RELATED FORMULATIONS

Latent world models such as DINO-WM (Zhou et al., 2025), V-JEPA 2-AC (Assran et al., 2025) and LeWM (Maes et al., 2026b) predict future features of a frozen or jointly trained encoder with a network whose output *replaces* the feature grid: absolute regression, usually rolled out one step at a time. Residual variants predict $\mathbf{Z}_k - \mathbf{Z}_0$, which keeps static patches cheap but still requires the network to write the full displacement of every moving patch (Proposition 2). Video prediction in pixel space has long used explicit motion (flow warping, dynamic filters, kernel-based transformations); SHIFTWm brings this structure to the feature space of foundation models, where features are

high-dimensional and semantically stable under motion, so that copying them is accurate. Unlike optical-flow pipelines, the transport is never supervised with flow: it is learned end-to-end from the feature forecasting loss, conditioned on future actions, and is used at every horizon directly from the observed grid.

A.5 EXTENDED RELATED WORK

Generative world models and model-based reinforcement learning. Pixel-space generative world models learn high-fidelity, action-controllable video simulators. NWM conditions a diffusion transformer on navigation actions (Bar et al., 2025); Cosmos and Cosmos-Predict2.5 provide large video foundation models for physical AI (NVIDIA, 2025a;b) that Cosmos Policy adapts for control (Kim et al., 2026); Ctrl-World, IRASim, UniSim and IWS target robot manipulation and policy evaluation (Guo et al., 2026b; Zhu et al., 2025; Yang et al., 2024; Wang et al., 2026b); and the Genie family generates interactive environments from video, image or text prompts (Bruce et al., 2024; Parker-Holder & Fruchter, 2025). Model-based reinforcement learning instead learns task-specific latent dynamics jointly with rewards and policies, from PlaNet and DreamerV3 to TD-MPC2, IRIS and DIAMOND (Hafner et al., 2019; 2025; Hansen et al., 2024; Micheli et al., 2023; Alonso et al., 2024). These systems optimise sample fidelity or online return with their own encoders and data, so they do not share our contract of a fixed frozen encoder, identical observations and actions, and targets in common feature coordinates. We therefore treat them as complementary rather than as compute-matched baselines.

World models for surgery and evaluation suites. Surgical world modelling is driven by generative video models: Endora simulates endoscopy video (Li et al., 2024), SurgWM learns latent-action-controllable surgical video (Koju et al., 2025), SurgVista targets long-horizon instrument-tissue dynamics (Pan et al., 2026), SWoMo combines neural and symbolic components for cataract surgery (Sivakumar et al., 2026), and Cosmos-H-Surgical learns surgical robot policies from videos through world modelling (He et al., 2025). Open-H-Embodiment releases paired video and kinematics across surgical platforms, together with a kinematics-conditioned surgical simulator (Open-H-Embodiment Consortium, 2026); SutureBot, JIGSAWS, Cholec80 and CholecT50 provide surgical demonstrations and task, skill and workflow annotations (Haworth et al., 2025; Gao et al., 2014; Twinanda et al., 2017; Nwoye et al., 2022), and SurgWMBench benchmarks instrument-motion planning (Liu et al., 2026). We use Open-H dVRK recordings to test kinematics-conditioned forecasting in feature space, a regime these pixel generators do not target. Robot datasets such as DROID, BridgeData V2 and Open X-Embodiment supply action-labelled video (Khazatsky et al., 2024; Walke et al., 2023; Open X-Embodiment Collaboration, 2024). WorldScore, WorldModel-Bench, EWMBench and Physics-IQ score generated video for controllability and physical plausibility (Duan et al., 2025; Li et al., 2025; Yue et al., 2025; Motamed et al., 2026), whereas stable-worldmodel standardises latent planning evaluation (Maes et al., 2026a). We follow the latter for control and report feature-space forecasting error for prediction.

Encoders. SHIFTWM applies to any frozen patch encoder, e.g. DINOv2 or DINOv3 (Oquab et al., 2024; Siméoni et al., 2025).

B EXTENDED METHOD

B.1 ARCHITECTURE

Architecture. Width $d=384$, 6 heads, MLP ratio 4, memory encoder $L_e=4$ layers of full self-attention over the $H \cdot N=768$ tokens of $H=3$ observed 16×16 grids. Query decoder: $L_d=4$ layers, each with self-attention over the $N=256$ query tokens, cross-attention to memory and an MLP, modulated by AdaLN-zero on c_k . Action embedding: a 2-layer MLP followed by a single-layer unidirectional GRU of width d . Transport: key dimension 64, window $w=7$ over $S=3$ source frames (147 candidates per query), learned identity bias initialised at $\beta=4$, and gate bias -3 . Correction head: LayerNorm, then a zero-initialised linear map to C , then a per-channel scale initialised at 1. Parameters: 21.75M (SHIFTWM) and 21.70M (Direct, AR, AR-TF).

B.2 PROOFS

Proof of Proposition 1. The conditional mean minimises expected squared error. Since $\mathbf{Z}_0$ is part of the context and $\mathbb{E}[\boldsymbol{\varepsilon}_k \mid \mathbf{P}_k, \mathbf{Z}_0, \mathbf{a}] = \mathbf{0}$, $\mathbb{E}[\mathbf{Z}_k \mid c] = \mathbb{E}[\mathbf{P}_k \mid c] \mathbf{Z}_0 + \mathbb{E}[\mathbb{E}[\boldsymbol{\varepsilon}_k \mid \mathbf{P}_k, c] \mid c] = \bar{\mathbf{P}}_k \mathbf{Z}_0$. Each row of $\mathbf{P}_k$ is a one-hot vector, so each row of $\bar{\mathbf{P}}_k$ is a probability vector supported on the same indices, hence on $\mathcal{W}(i)$ for row i . For $g = 1$ and $\mathbf{r} = \mathbf{0}$, Equation (2) gives $\hat{\mathbf{Z}}_{k,i} = \sum_{j \in \mathcal{W}(i)} \pi_{k,i}(j) \mathbf{Z}_{0,j}$. Softmax weights with logits $\ell_j = \log \bar{P}_{k,ij}$ on the support and $\ell_j \rightarrow -\infty$ elsewhere converge to row i of $\bar{\mathbf{P}}_k$, so $\hat{\mathbf{Z}}_k \rightarrow \bar{\mathbf{P}}_k \mathbf{Z}_0$. With $S > 1$ source frames the same argument applies to a motion model whose rows select patches of any observed frame. $\square$

Proof of Proposition 2. An additive head outputs $\mathbf{Z}_0 + \mathbf{R}$; matching $\mathbf{P} \mathbf{Z}_0$ requires $\mathbf{R} = (\mathbf{P} - \mathbf{I}) \mathbf{Z}_0$, whose i -th row is $\mathbf{z}_{0,\pi(i)} - \mathbf{z}_{0,i}$; summing squared row norms gives the claim. For SHIFTWM, the one-hot transport onto $\pi(i)$ (the limit above) with $g = 1$ reproduces $\mathbf{P} \mathbf{Z}_0$ with $\mathbf{r} = \mathbf{0}$. $\square$

Proof of Proposition 3. Let $e_j = \|\hat{\mathbf{Z}}_j - \mathbf{Z}_j\|$ for the recursive rollout $\hat{\mathbf{Z}}_j = f(\hat{\mathbf{Z}}_{j-1})$ and $\mathbf{Z}_j = f(\mathbf{Z}_{j-1}) + \boldsymbol{\eta}_j$ with $\|\boldsymbol{\eta}_j\| \leq \delta$. Then $e_j \leq \|f(\hat{\mathbf{Z}}_{j-1}) - f(\mathbf{Z}_{j-1})\| + \|\boldsymbol{\eta}_j\| \leq L e_{j-1} + \delta$ with $e_0 = 0$, so $e_k \leq \delta \sum_{j=0}^{k-1} L^j$. A direct predictor computes $\hat{\mathbf{Z}}_k$ from observed inputs only, so its error at horizon k does not depend on its predictions at other horizons. $\square$

B.3 PLUG-IN HEADS FOR PUBLISHED WORLD MODELS

For a published backbone with prediction $\mathbf{P}_k$, the head outputs $\hat{\mathbf{Z}}_k = (1 - g_k) \mathbf{P}_k + g_k \mathbf{T}_k$, where $\mathbf{T}_k$ is the local 7×7 soft transport (with identity bias) of *observed* encoder tokens, and the gate $g_k = \sigma(\mathbf{w}^\top \mathbf{h} + b)$ has $\mathbf{w} = \mathbf{0}$, $b = -4$ at initialisation, so training starts numerically from the unmodified backbone. Nothing else in the backbone, loss or schedule changes. **V-JEPA 2-AC.** Queries come from the predictor’s normalised hidden state for each output block (1024-D), keys and values from the layer-normalised ViT-g/16 tokens (16×16 , 1408-D) of the last $S=3$ measured frames, with a learned per-source key offset (0.16M parameters). The fine-tuning objective is the released one (teacher-forced L1 plus rollout L1). During rollouts the predictor consumes $\hat{\mathbf{Z}}_k$ as its own prediction, while $\mathbf{T}_k$ only ever reads measured tokens. **DINO-WM.** We subclass the official ViTPredictor (commit 0a9492f, otherwise unmodified): queries come from the predictor’s output token, keys and values from its input token at the same slot, on the 14×14 DINOv2 patch grid ($S=1$); proprioceptive and action channels are left untouched. Keys and values are therefore measured tokens only at the first rollout step: afterwards the input token at that slot is DINO-WM’s own previous prediction, so transport moves predicted features. We hypothesise, without proof, that this contributes to the Wall regression. The extra parameters are drawn from a private random generator, so data order, initialisation and dropout of the base predictor are identical in both arms. Planning uses the official DINO-WM planner at a reduced budget that is identical for both arms.

C EXTENDED EXPERIMENTAL DETAILS

C.1 BENCHMARKS AND EVALUATED TASKS

Table 5 states, for every benchmark, what the data show and what the world model is asked to do. In all forecasting benchmarks the task is the same: given the last $H=3$ observed frames (encoded to frozen DINOv2 patch grids), the actions between them and the next K action blocks, predict the patch features of the next K frames, without seeing them. The benchmarks differ in embodiment, scene and action space. In the planning benchmarks the world model is used inside a planner, and the task is to reach a goal image.

C.2 DATASETS AND SPLITS

DROID. We use 24 official full-release shards (22.0 GB) containing 1,126 episodes from 14 sites (327,498 native steps, 64,826 action blocks; 316/59/58 recording sessions in train/val/test). Splits are assigned by recording session with a salted hash, so no session appears in two splits. Frames

Table 5: **Benchmarks and evaluated tasks.** Forecast: predict K future feature grids from $H=3$ observed frames and K future action blocks.

Benchmark	What is recorded	Evaluated task	Metric (test unit)
DROID	Franka arm doing everyday manipulation in real homes, labs and offices; exterior camera	forecast $K=10$ steps (3.3 s) given commanded end-effector poses and gripper	feature MSE, skill (held-out recording sessions)
DROID cam 2	the same episodes seen from the second exterior camera	zero-shot transfer of the camera-1 model	as above
Open-H Hamlyn	dVRK arms on 7 surgical training tasks: knot tying, needle grasp and handover, peg transfer, suturing (2), tissue lifting, tissue retraction; scene camera	forecast $K=10$ steps (3.3 s) given both arms' pose and gripper targets	feature MSE, skill (held-out episodes, per task)
BridgeData V2	WidowX arm on tabletop and kitchen tasks (Open-X subset)	forecast $K=10$ steps (4 s) given end-effector deltas and gripper	feature MSE, skill (held-out episodes)
RT-1 (Fractal)	mobile manipulator picking, placing, opening drawers in offices (Open-X subset; base-motion episodes removed)	forecast $K=10$ steps (3.3 s) given end-effector deltas and gripper	feature MSE, skill (held-out episodes)
Language-Table	xArm pushing coloured, textured blocks on a table under language instructions; fixed camera, 640×360 (Open-X subset)	forecast $K=10$ steps (1 s) given 2-D end-effector setpoint deltas	feature MSE, skill (held-out episodes)
IWS PushT / Box / Rope	real bimanual recordings of pushing a T-block, handling a box, and manipulating a rope (RLA-WM release)	forecast $K=12$ steps (2 s) given the commanded joint targets	feature MSE on the 200 official validation handles per task
LeWM PushT / TwoRoom / Reacher	simulated pushing of a T-block, navigation through a doorway between two rooms, and a two-link arm reaching	plan actions with CEM so the predicted future matches a goal image	success rate over 50 episodes $\times$ 3 planner seeds
DINO-WM PushT / Wall	the DINO-WM releases of PushT and a point agent navigating through a door in a wall	latent prediction error; planning with the official DINO-WM planner	latent error; success rate

Table 6: **Datasets.** Step = one model time step. All splits are disjoint at the unit shown.

Dataset	Domain	Episodes (train/val/test)	Step	Action dim	Split unit	License
DROID (24 shards)	real Franka, 3 cams	851 / 143 / 132	0.33 s	35	session	CC-BY-4.0
Open-H Hamlyn (7 tasks)	surgical dVRK	708 / 152 / 153	0.33 s	80	episode	CC-BY-4.0
IWS PushT / Box / Rope	real bimanual	510-512 / 90 / 200 handles	0.17 s	20 / 70 / 40	recording	pending
BridgeData V2 (subset)	real WidowX	518 / 57 / 159	0.4 s	14	official split	CC-BY-4.0
RT-1 Fractal (subset)	real mobile robot	8,657 / 942 / 1,101	0.33 s	7	episode	CC-BY-4.0
Language-Table (subset)	real xArm pushing	2,079 / 246 / 264	0.1 s	2	episode	CC-BY-4.0
PushT / TwoRoom / Reacher	simulation	pending	5 env steps	10	episode	MIT

are subsampled every 5 native steps (15 Hz $\rightarrow$ 3 Hz), and each action block concatenates the 5 commanded Cartesian poses and gripper positions ($5 \times 7=35$).

Open-H-Embodiment Hamlyn. Seven dVRK training tasks, released under CC-BY-4.0 in LeRobot format. We use the `observation.images.color` stream, which the release labels “endoscope” but which is an Intel RealSense scene camera (848×480). One step is 1/3 s: stride 10 at 30 Hz, or stride 5 for the 15 Hz tissue-lifting task. Each action block holds five commands at 15 Hz, each an 8-D pose-and-gripper target for each arm ($5 \times 16=80$). Episodes shorter than 14 steps (6 of 1,019) are removed. Splits are 70/15/15 per task, assigned by a hash of the episode id.

IWS. Real bimanual recordings from the RLA-WM release (PushT, Box, Rope; 640×480 at 30 fps). One model step is 5 native command rows (1/6 s), so the official 60-row horizon is $K=12$ steps; each action block concatenates the 5 commanded joint-target rows. Train and validation use the upstream training recordings (episode-disjoint, 15% validation by a hash of the episode id); the test set is exactly the 200 official RLA-WM validation handles per task, drawn from 10 reserved recordings.

BridgeData V2 and RT-1 (Fractal). Pinned Open-X RLDS shards, primary camera, full frame resized to 224. For Bridge (5 Hz) one model step is 2 native steps (0.4 s) and each block holds two 7-D actions (end-effector translation and rotation deltas, gripper); for RT-1 (3 Hz) one step is one native step (1/3 s) with the same 7-D action, dropping the base-motion dimensions and skipping the $\approx 1\%$ of episodes where the base moves.

Language-Table. The real-robot Language-Table release (Lynch et al., 2023) in Open-X RLDS form: an xArm with a cylindrical end effector pushes rigid, coloured blocks on a wooden board,

seen by a fixed camera (640×360 , resized to 224 like Bridge and RT-1 and shown at its native aspect). We fetch 14 of the 1,024 pinned shards (6.0 GB, 6,161 episodes). The Open-X sheet lists 10 Hz control, so one model step is one native step (0.1 s) and the action is the 2-D delta end-effector setpoint. We do not subsample further: the episodes are short hindsight-labelled segments of longer continuous play (median 12 native steps), so a 0.3 s step would leave only 331 episodes long enough for $H+K=13$ steps, and the end effector already moves ≈ 2.7 cm per native step (median). Episodes shorter than 14 steps are dropped (3,572), leaving 2,589, split 10%/9%/81% into test/val/train by a salted hash of the episode id. Open-X ships only a `train` split, so every id keeps that name (`train00160_052`: shard 160, record 52) and we carve validation and test from it. The split is by segment: the release has no id of the source play episode, so segments of one play session can fall into different splits and share a scene layout. In our subset we find no direct continuations: for only 1 of 264 test segments does a training segment start or end with a frame closer to the test segment’s boundary frame than the median one-step frame change, but source-level disjointness is not guaranteed.

Planning suites. PushT, TwoRoom and Reacher expert datasets from the LeWM release (HDF5). Episodes are split disjointly into train and validation sets for world-model training. Evaluation follows the official protocol (Appendix C.5).

C.3 BASELINES

Matched predictors. All learned baselines share SHIFTWM’s memory encoder, decoder, width, depth, optimiser and schedule, and differ only in the output head and rollout. This isolates the contribution of transport from those of capacity and spatial communication. Direct has the same cross-patch attention as SHIFTWM and differs only in lacking the transport and gate.

DINO-WM-style (AR-TF). Following Zhou et al. (2025), the predictor regresses absolute next-step patch features with teacher forcing and is rolled out autoregressively at test time. We use the same frozen DINOv2 patches at native resolution.

External reference: V-JEPA 2-AC. We evaluate the released V-JEPA 2-AC (Assran et al., 2025) (ViT-g encoder and 300M action-conditioned predictor, trained on DROID) zero-shot on exactly our test windows, with its own preprocessing, action convention and autoregressive rollout. Because feature spaces differ, we compare the fraction of persistence error each model removes in its own space (Table 7); we also fine-tune it on our training split with and without the SHIFTWM head under an identical budget. Its training data likely include our test trajectories.

Table 7: **External models on the same DROID test windows:** % of persistence error removed (skill $\uparrow$), each in its own feature space.

Model	Feature space	Skill (%) [95% CI]
V-JEPA 2-AC (zero-shot, ViT-g, 1.3B)	V-JEPA 2 ViT-g	4.1 [0.6, 7.3]
V-JEPA 2-AC fine-tuned on our split (2 seeds)	V-JEPA 2 ViT-g	24.6
V-JEPA 2-AC fine-tuned + SHIFTWM head (2 seeds)	V-JEPA 2 ViT-g	36.4
AR-TF (DINO-WM-style)	DINOv2-S	-10.4
AR	DINOv2-S	17.4
Direct	DINOv2-S	21.6
SHIFTWM (ours)	DINOv2-S	27.5

C.4 TRAINING AND COMPUTE

Training. AdamW ($\beta=(0.9, 0.95)$, weight decay 0.05), peak learning rate 3×10^{-4} with 800 warm-up steps and cosine decay, 16k steps, batch 48, bf16 autocast, gradient clipping at 1.0. Validation MSE (averaged over horizons, windows at stride 2) is computed every 2k steps, and the best checkpoint is used for testing. AR is trained through its full 10-step rollout. AR-TF is trained with teacher forcing and rolled out at test time. All features for a split are kept in GPU memory as float16 and windows are gathered by index.

Compute. All experiments ran on NVIDIA H200 GPUs, packing at most two learned predictors per GPU. pending

Table 8: **Hyperparameters** (shared by all learned predictors unless noted).

Width / heads / MLP ratio	384 / 6 / 4	Optimiser	AdamW, $\beta=(0.9, 0.95)$
Memory encoder layers L_e	4	Peak LR / warm-up	3×10^{-4} / 800 steps
Query decoder layers L_d	4	Schedule	cosine, 16k steps
History H / horizon K	3 / 10	Batch size	48 windows
Transport window w / sources S	7 / 3	Weight decay / clip	0.05 / 1.0
Key dim / identity bias init	64 / 4.0	Precision	bf16 autocast
Gate bias init	-3	Selection	best val. MSE (every 2k steps)
Action-contrastive λ / margin	1.0 / 0.05 (planning)	Seeds	0, 1, 2

C.5 PLANNING PROTOCOL

Closed-loop planning follows the official LeWM and stable-worldmodel evaluation (Table 9). Every predictor, including the released LeWM checkpoint, is wrapped in the same pluggable interface and driven by the same CEM planner, goal sampling, budget and success criteria; only the world model changes. The planning cost is the squared distance between the final predicted latent and the goal latent. Appendix D.6 checks that the harness reproduces the published LeWM results.

Table 9: **Official planning protocol** (le-wm@8edfeb33, stable-worldmodel@4821c8e6).

Planner	CEM, 300 samples $\times$ 30 iterations, 30 elites, initial variance 1, warm start
Horizon / replanning	5 action blocks of 5 env steps; replan every 25 steps
Episodes / seeds	50 episodes per planner seed; seeds 42, 43, 44
Goal	dataset frame 25 steps later in the same trajectory
Budget	50 environment steps
Cost	squared distance between final predicted latent and goal latent
Success	PushT: position < 20 px and angle $< \pi/9$; TwoRoom: < 16 px; Reacher: joint match

C.6 STATISTICAL PROCEDURES

Each test episode contributes the mean of its windows (stride 2). For each method and seed we average over episodes, and tables report the mean over three training seeds. For a paired comparison between methods A and B, we compute per-episode differences (averaged over seeds) and draw 10,000 bootstrap resamples of episodes to form a percentile 95% interval. For DROID, we resample sessions instead of episodes to respect the session structure. Primary contrasts (SHIFTWM against each baseline on each dataset) are Holm-adjusted within each dataset.

D EXTENDED RESULTS

D.1 FULL FORECASTING RESULTS

Table 10: **Per-horizon test MSE on DROID** (mean over 3 seeds).

Method	$k=1$	2	3	4	5	6	7	8	9	10
Persistence	0.108	0.165	0.205	0.235	0.258	0.278	0.294	0.307	0.317	0.326
AR-TF (DINO-WM-style)	0.099	0.151	0.195	0.234	0.269	0.302	0.333	0.362	0.390	0.417
AR (rollout-trained)	0.101	0.146	0.174	0.195	0.211	0.226	0.238	0.248	0.256	0.264
Direct (cross-attn)	0.095	0.137	0.165	0.185	0.202	0.215	0.227	0.236	0.243	0.250
SHIFTWM (ours)	0.089	0.127	0.152	0.171	0.186	0.199	0.209	0.218	0.225	0.231

pending

D.2 ADDITIONAL ABLATIONS

Training recipes. Because halving training to 8k steps barely changes test error (Table 13), we tested whether stronger regularisation or more data helps (Table 13): training on both exterior cameras with EMA weights, dropout and memory-token dropout, and adding explicit correlation (cost-volume) features between consecutive observed grids to every method. pending pending

Table 11: **Open-H Hamlyn test MSE per surgical task** (mean over horizons and 1 training seed(s)). Table 1 averages over episodes, so tasks are weighted by their number of test episodes.

Method	Knot	Needle	Peg	Sut. 1	Sut. 2	Lift	Retract
Persistence	0.600	0.522	0.350	0.376	0.471	0.546	0.575
AR-TF (DINO-WM-style)	0.401	0.353	0.246	0.223	0.297	0.335	0.314
AR (rollout-trained)	0.329	0.304	0.201	0.187	0.248	0.254	0.247
Direct (cross-attn)	0.322	0.304	0.201	0.184	0.244	0.257	0.253
SHIFTWM (ours)	0.312	0.297	0.194	0.177	0.237	0.241	0.239

Table 12: **IWS per task** on the official RLA-WM handles.

Method	PushT		Box		Rope	
	MSE	LPIPS	MSE	LPIPS	MSE	LPIPS
Persistence	pending	pending	pending	pending	pending	pending
AR	pending	pending	pending	pending	pending	pending
Direct	pending	pending	pending	pending	pending	pending
SHIFTWM	pending	pending	pending	pending	pending	pending

D.3 PLUG-IN TRAINING DYNAMICS

This subsection supports the plug-in contribution (Section 5.2). Figure 9 shows both fine-tuning arms over their full 3,000-step budget. With the head, validation error is lower at every evaluation step for both seeds. The gate starts at 0.018, so the model begins as the released predictor, and opens to about 0.55 within the first 250 steps, rising to about 0.65: the backbone learns to route most of its forecast through transport of measured tokens. Both arms reach their best validation error early and then slowly overfit our training split; the test numbers use the best validation checkpoint of each run. The skill gain holds at every horizon and grows from +11.5 points at $k=1$ to +13.1 at $k=10$.

D.4 DECODED-PIXEL METRICS

For each dataset we train one feature-to-RGB decoder on true training features (L1 + LPIPS) and apply it unchanged to every method’s forecasts, so differences reflect the predicted features only.

pending

D.5 PROBING PREDICTED FEATURES

Ridge probes map mean- and 4×4 -pooled features to the commanded DROID end-effector position or the measured Hamlyn instrument-tip positions (both arms pooled). They are fit on true training features, with the ridge strength chosen on validation, and are evaluated on true versus predicted test features at $k=10$.

D.6 PLANNING RESULTS AND REPRODUCTION CHECK OF THE HARNESS

Forecast error bounds planning regret. Let the planner score an action sequence $\mathbf{a}$ by $\hat{c}(\mathbf{a}) = \|\hat{\mathbf{z}}_T(\mathbf{a}) - \mathbf{z}_g\|^2$, where $\mathbf{z}_g$ is the goal encoded by the same frozen encoder, and let $c(\mathbf{a})$ use the true future $\mathbf{z}_T(\mathbf{a})$. If $\|\hat{\mathbf{z}}_T - \mathbf{z}_T\| \leq \varepsilon$ and $\|\mathbf{z}_T - \mathbf{z}_g\| \leq R$, then $|\hat{c} - c| = |2\langle \hat{\mathbf{z}}_T - \mathbf{z}_T, \mathbf{z}_T - \mathbf{z}_g \rangle + \|\hat{\mathbf{z}}_T - \mathbf{z}_T\|^2| \leq 2R\varepsilon + \varepsilon^2$. A plan $\hat{\mathbf{a}}$ within η of the best predicted cost therefore satisfies $c(\hat{\mathbf{a}}) \leq \min_{\mathbf{a}} c(\mathbf{a}) + 4R\varepsilon + 2\varepsilon^2 + \eta$: lower forecast error tightens the regret bound, but only for errors along directions that change the ranking of candidate plans, which is why we also measure action sensitivity (Appendix D.7).

Reproduction check. With the released LeWM PushT checkpoint and the official protocol, our harness reaches 45/50, 44/50 and 43/50 successes for planner seeds 42/43/44 (mean 88%). An adapter check confirms that our pluggable predictor interface produces exactly the native LeWM planning costs (maximum absolute difference 0). The LeWM paper reports 96 ± 2.8 , averaged over training seeds, whereas the released checkpoint is a single training seed. On TwoRoom and Reacher the released checkpoints reach 43/41/44 and 42/44/41 successes (85.3% and 84.7%), against 87 and 86 reported by [Maes et al. \(2026b\)](#). Planning takes 0.41 s (PushT) and about 1.5 s (TwoRoom,

Table 13: **Training recipes and add-ons** on DROID (*validation* MSE, seed 0; recipes were chosen on validation, the test split is scored once). Early stopping: SHIFTFWM and Direct stopped at 3k/4k steps, AR ran 8k; AR-TF and some correlation-feature rows are incomplete (*pending*).

Recipe	AR-TF	AR	Direct	SHIFTFWM
base (16k steps)	0.299	0.224	0.214	0.200
base, short (8k steps)	0.297	0.224	0.216	0.202
+ 2nd camera, EMA, dropout	pending	0.228	0.222	0.201
+ correlation features	pending	pending	0.222	0.213

Pending results. Validation MSE vs. transport window size $w \in \{3, 5, 7, 11, \text{global}\}$ and number of source frames $S \in \{1, 3\}$.

Figure 8: **Transport window and memory size.**

Reacher) per CEM plan on one H200. As an independent check, LeWM’s own evaluation code (an earlier release of the planning library) gives 46/50 on PushT and 39/50 on Reacher for the same checkpoint and seed 42, against 45/50 and 42/50 in our harness; the Reacher difference of 3 episodes is within planner-seed variation (41–44 in our runs) but we report it for transparency.

D.7 ACTION SENSITIVITY AND EFFICIENCY

Accuracy versus action sensitivity. A forecast that stays close to $\mathbf{Z}_0$ can score well on average while barely responding to the commanded actions, and a planner then cannot rank candidate action sequences. We measure this directly as the rate at which true future actions yield lower error than actions from another episode (Figure 10, left). Anchored predictors can reach low open-loop error while reacting weakly to actions; the action-contrastive objective moves SHIFTFWM toward high accuracy *and* high sensitivity. *pending*

Efficiency. Table 16 compares parameters, compute per forecast step and latency. Transport adds *pending* parameters. Because SHIFTFWM and Direct decode all K steps in parallel, a K -step forecast needs one encoder pass, whereas recursive predictors need K (*pending*).

E EXTENDED ANALYSIS AND DISCUSSION

E.1 ANALYSIS DETAILS

Each analysis below states the contribution it supports: the method (transport carries moving content and the identity path keeps static content), the motivating motion model (Propositions 1 and 2), or the evaluation of where the gains arise.

Feature-space geometry (supports Proposition 1). Figure 12 is the data counterpart of Figure 3. Panel (a) shows, for one patch at a time, the $Sw^2=147$ observed candidate features of its transport window projected to 2-D by PCA, sized by the learned transport weight, together with the observed feature at the same location, the true future and the three forecasts. The patches illustrate the mechanism where it matters most: among moving, high-gate ($g>0.5$) patches with above-median change, the three (from different episodes) with the largest advantage of SHIFTFWM over the better baseline. They are selected, not typical; the typical behaviour is given by the aggregates. Panels (b) and (c) aggregate over all moving patches of 3,923 windows from 130 test episodes. The oracle move picks, for each patch, the single observed candidate closest to the true future; it is a model-free reference, not an achievable predictor, since it uses the future to choose. Gain recovered is

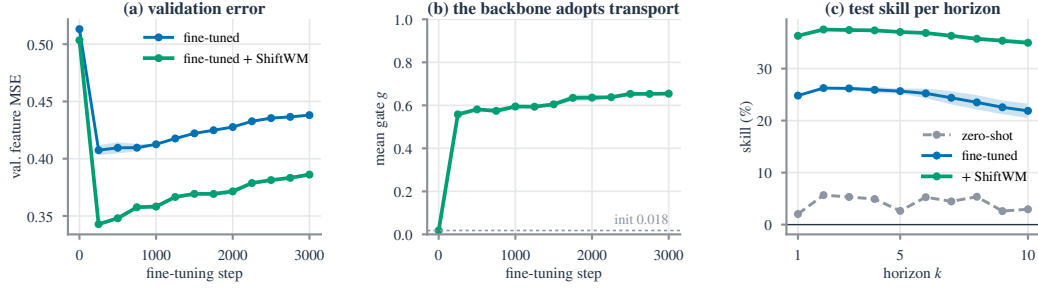


Figure 9: V-JEPA 2-AC with and without the SHIFTWM head (DROID). (a) Validation error while fine-tuning (2 seeds). (b) Mean gate. (c) Test skill per horizon.

Table 14: Decoded-pixel quality at $k=10$. One decoder per dataset for all methods; the first row decodes true features.

Method	DROID			Open-H Hamlyn		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Decoder on true features (upper bound)	pending	pending	pending	pending	pending	pending
Persistence	pending	pending	pending	pending	pending	pending
AR-TF (DINO-WM-style)	pending	pending	pending	pending	pending	pending
AR	pending	pending	pending	pending	pending	pending
Direct	pending	pending	pending	pending	pending	pending
SHIFTWM	pending	pending	pending	pending	pending	pending

$(\text{err}_{\text{stay}} - \text{err}_{\text{model}}) / (\text{err}_{\text{stay}} - \text{err}_{\text{oracle}})$. Paired episode-bootstrap 95% intervals of the moving-patch error difference at $k=10$ exclude zero against both Direct and AR.

Anatomy of the gains (supports the method). Figure 13 uses all 3,923 test windows. In (a,b) every patch at every horizon is assigned to a decile of its true change $\|\mathbf{z}_{k,i} - \mathbf{z}_{0,i}\|^2$ (deciles pooled over horizons), and each cell reports $1 - \text{err}_{\text{SHIFTWM}} / \text{err}_{\text{baseline}}$; the 10×10 grid is drawn as filled contours after cubic interpolation, with the maximum marked. Against Direct the gain is largest for moderate-to-large motion at long horizons; against AR it is largest on nearly static patches, where the recursive rollout accumulates error on content that should not change. (c) The forecast departs from $\mathbf{Z}_0$ as $g(\mathbf{T} - \mathbf{Z}_0) + \mathbf{r}$ exactly; on moving patches the mean gate rises from 0.51 at $k=1$ to 0.85 at $k=10$, and the moved term carries 87% of the squared departure on average, so the correction remains a small residual. (d) On moving patches SHIFTWM recovers a larger share of the oracle-move gain than both baselines at every horizon, reaching 79% at $k=10$ against 71% (Direct) and 69% (AR).

Transport versus optical flow. Figure 14 compares the expected source displacement of each patch with RAFT optical flow (Teed & Deng, 2020) pooled to the patch grid. pending

E.2 SEGMENTATION VIEW OF THE FORECASTS

Does the moving arm land in the right place? (Evaluation beyond feature error.) Feature error does not show *where* a forecast puts the arm, so we read a segmentation out of every forecast (Figure 15). SAM 2.1 (Ravi et al., 2025) tracks the robot arm (the instruments on Hamlyn) through the true future frames, and each forecast patch is labelled arm or background by nearest-neighbour transfer from the observed frame alone (protocol below). *Placement* is the distance between the centroid of the cells a forecast labels as arm and the centroid of the true mask at $t+k$. On the half of the 1,332 held-out DROID windows where the arm moves most, the mean placement error over k is 12.0 px for SHIFTWM, 12.2 for Direct, 13.9 for AR and 17.6 for persistence (320×180 frame): SHIFTWM removes **+2.1 px** relative to AR (95% CI [+1.5, +2.7]), but only +0.6 px relative to Direct ([+0.0, +1.1]), a marginal difference; at $k=10$ SHIFTWM and Direct tie (16.7 vs. 16.8 px). On Hamlyn all learned models place the instruments within a pixel of each other (SHIFTWM 12.3, Direct 12.4, AR 12.7 px; paired reductions +0.2 [+0.0, +0.4] and +0.3 [+0.1, +0.6] px), all far ahead

Table 15: **Robot state decoded from predicted features** at $k=10$ by linear probes (DROID: end-effector, cm; Hamlyn: instrument tips, mm).

Method	DROID EE position		Hamlyn tip position	
	MAE↓	Pearson r ↑	MAE↓	Pearson r ↑
True future features (upper bound)	pending	pending	pending	pending
Persistence	pending	pending	pending	pending
AR-TF (DINO-WM-style)	pending	pending	pending	pending
AR	pending	pending	pending	pending
Direct	pending	pending	pending	pending
SHIFTWM	pending	pending	pending	pending

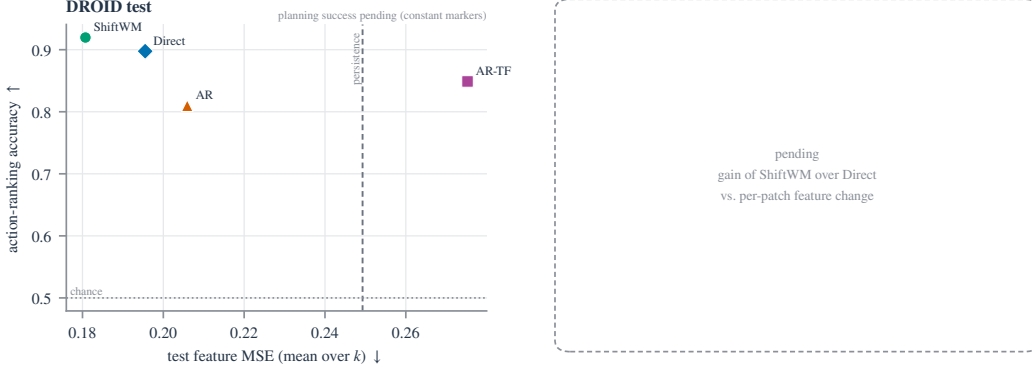


Figure 10: **Left:** forecast error vs. action-ranking accuracy (marker size: planning success). **Right:** gain of SHIFTWM over Direct by amount of motion.

of persistence (16.6 px). Overlap tells the same story: on moving DROID windows the arm read out of SHIFTWM has IoU **+2.0** points above Direct ($[+1.2, +2.7]$) and **+4.2** above AR ($[+3.3, +5.1]$); on Hamlyn it ties Direct ($+0.1, [-0.2, +0.4]$; Table 18). The examples in Figure 15a are selected for a large SHIFTWM placement advantage and illustrate what that advantage looks like; the claims rest on the averages.

Protocol. We ask whether the moving part of the scene lands where it should in each forecast, without decoding pixels. *Reference masks.* On the observed frame t (the last history frame) of every held-out window (test split; the planning suites have no test split, so their validation split is used; stride 2 over all episodes, and an evenly spaced subset of 1,500 windows where there are more), Grounding DINO (Liu et al., 2024) detects a fixed per-dataset text prompt (DROID, Bridge, RT-1: “a robot arm”; Hamlyn: “a surgical instrument”, up to two non-overlapping boxes; Language-Table: “a robot arm. a block.”, up to two boxes; IWS: the manipulated T block, box or rope). Its top box prompts SAM 2.1 (Hiera-L) (Ravi et al., 2025), which segments frame t and *tracks* the mask through the true future frames $t+1, \dots, t+K$ with its video memory. Frames are segmented at their native aspect ratio; each mask keeps its largest connected component and any component with $\geq 25\%$ of its area. For the rendered planning suites a text prompt is unreliable, so an exact colour threshold of the agent/object is used instead. *Quality control.* A window is kept only if the detection score is ≥ 0.3 , the mask covers 3–128 of the 256 patches at t , the track is never lost and the mask area never changes by more than $3\times$ between consecutive frames; on DROID this keeps 1,332 of 1,500 windows (rejected: 104 without a confident detection, 45 with an arm too small or too large at t , 8 lost tracks, 11 area jumps), and on Hamlyn 1,017 of 1,500. *Reading a mask out of a forecast.* Masks are area-averaged onto the 16×16 patch grid (a patch is foreground if $\geq 50\%$ covered). Each patch of a forecast $\hat{\mathbf{Z}}_k$ is labelled by nearest-neighbour transfer from the observed frame only: it is foreground if its most similar (cosine) foreground patch of $\mathbf{Z}_0$ is more similar than its most similar background patch. This labeller was fixed in a DROID pilot as the one with the highest IoU when applied to the *true* future features, a criterion that involves no forecast (on DROID at $k=10$: nearest-neighbour 0.70, k -means prototypes 0.60, single prototypes 0.42). *Metric.* IoU between the forecast mask and the tracked SAM mask of the true future frame, per horizon and averaged over k , on all windows and on *moving* windows (overlap of the masks at t and $t+K$ below the median). CIs

Table 16: **Efficiency** of a 10-step forecast on one H200 (batch 1, bf16).

Predictor	Params (M)	GFLOPs / 10 steps	Latency (ms)	Train GPU-h
AR-TF / AR	21.70	pending	pending	pending
Direct	21.70	pending	pending	pending
SHIFTWM	21.75	pending	pending	pending

Table 17: **Where the gains come from** (DROID test, feature MSE ↓ by region; skill = error removed vs. persistence). Training seeds: AR s0; AR-TF s1; Direct s0, s1; SHIFTWM s0 (Table 1: mean over all seeds), computed on earlier checkpoints of the same recipe.

Method	moving patches	static patches	all	skill ↑
Persistence	0.726	0.090	0.249	0.0%
AR-TF (DINO-WM-style)	0.592	0.161	0.268	-7.6%
AR (rollout-trained)	0.499	0.105	0.203	18.4%
Direct (cross-attn)	0.484	0.097	0.194	22.3%
SHIFTWM (ours)	0.449	0.088	0.179	28.3%

are 95% bootstrap intervals over episodes; gains are paired differences per episode. *Placement.* The predicted centroid is the mean of the centres of the cells labelled arm; the true centroid is the coverage-weighted mean of cell centres under the tracked SAM mask; distances are in pixels of the displayed native-aspect frame (DROID 320×180, Hamlyn 396×224). At a given k , windows in which any compared method labels no cell are dropped for all methods (paired; this affects at most 1% of DROID and 3% of Hamlyn windows). *Examples* show where SHIFTWM does best, by a fixed rule: among QC-passing moving windows in which SHIFTWM’s $k=10$ placement error is below its median and the labeller applied to the *true* future features is at least median-accurate (a method-independent sanity check), the two (from distinct episodes) with the largest $k=10$ placement advantage $\min(\text{Direct}, \text{AR}) - \text{SHIFTWM}$. All methods are always shown on the same window and the same frame. The examples are not representative; averages and paired CIs are in Tables 18 and 19. Checkpoints are the final ones used in the main tables.

Results. Table 19 lists placement error and Table 18 IoU per dataset and method (Figures 16 and 17 show both example windows per dataset). On DROID, SHIFTWM has the highest IoU at every reported horizon, on all and on moving windows, with paired CIs that exclude zero against both Direct and AR. On Hamlyn the instruments move less within ten steps and all three learned predictors are within half an IoU point: SHIFTWM ties Direct and is marginally ahead of AR; the segmentation view does not separate the methods there. Datasets without results are marked pending.

Caveats. (i) The readout measures where a forecast puts content that *looks like* the observed foreground; it cannot credit content that was not visible at t . (ii) The labeller is imperfect: applied to the true future features it reaches IoU 0.70 on DROID at $k=10$, so absolute values are lower bounds on localisation quality; on Hamlyn several forecasts slightly exceed this reference, because smoother forecasts can be labelled more easily than the true features. (iii) SAM 2.1 occasionally includes a grasped object or tool-attached structure in the tracked mask; QC removes lost or exploding tracks but not such merges. On Hamlyn most rejected windows (363 of 483) are close-ups in which the instruments cover more than half of the frame. (iv) One seed per method.

Table 18: **Segmentation view: IoU of the foreground read out of each forecast** (horizon-averaged, $k=5$ and $k=K$; all and moving windows). **Green:** paired 95% CI of SHIFTWM minus *both* Direct and AR excludes zero.

Dataset	Method	all windows			moving windows		SHIFTWM gain, moving (IoU pts)	
		avg. over k	$k=5$	$k=K$	avg. over k	$k=K$	vs Direct	vs AR
DROID	Persistence	0.617	0.612	0.536	0.486	0.335		
	AR	0.628	0.624	0.551	0.535	0.427		
	Direct	0.644	0.638	0.576	0.557	0.454		
	SHIFTWM	0.654	0.651	0.592	0.576	0.482	+2.0 [+1.2, +2.7]	+4.2 [+3.3, +5.1]
Hamlyn	Persistence	0.742	0.743	0.700	0.646	0.569		
	AR	0.774	0.775	0.746	0.713	0.665		
	Direct	0.777	0.778	0.750	0.717	0.672		
	SHIFTWM	0.775	0.777	0.749	0.716	0.673	+0.1 [-0.2, +0.4]	+0.4 [+0.1, +0.8]

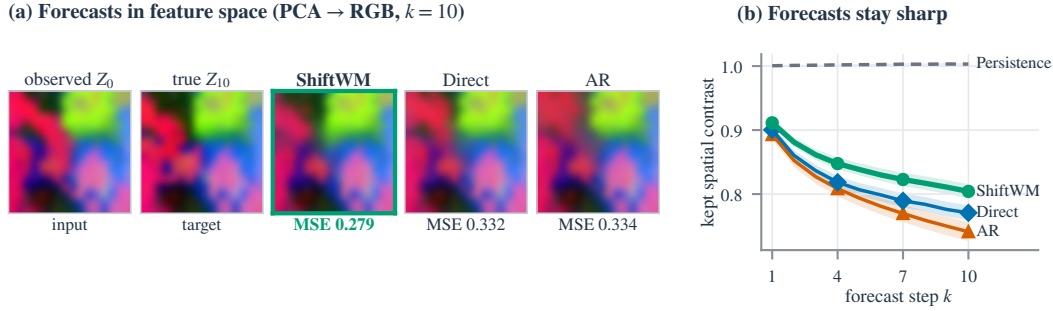


Figure 11: **Forecasts in feature space.** (a) Held-out forecasts at $k=10$ (shared PCA to colour). (b) Spatial contrast kept by each forecast, relative to the truth (95% CIs over test episodes).

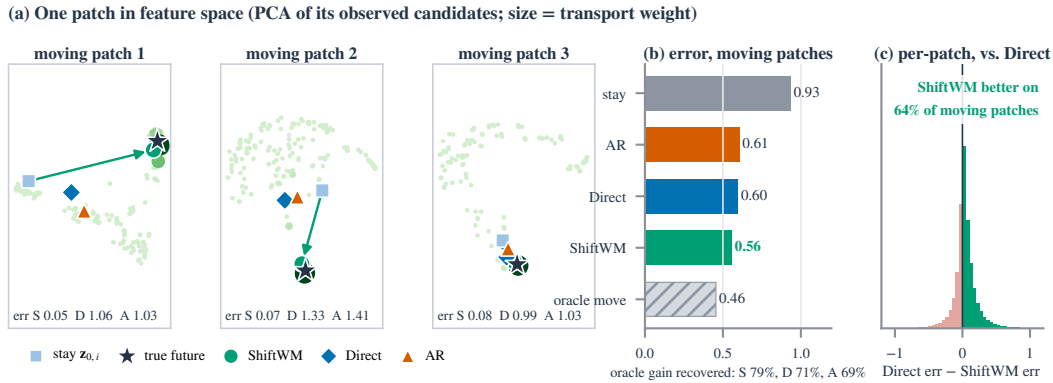


Figure 12: **Feature-space geometry on held-out DROID ($k=10$).** (a) Three moving patches (fixed selection rule) in a 2-D PCA of their observed candidates. (b) Moving-patch error, with the model-free oracle move. (c) Per-patch error difference to Direct.

E.3 QUALITATIVE RESULTS

Example selection. Illustrations in the main paper use selected held-out windows, and every method is always shown on the same window. The teaser strip, the bottom row of Figure 2 and Figure 11 use the DROID test window (among the 30% with the most motion) with the largest step-10 advantage of SHIFTWM over the better of Direct and AR; the win maps of Figure 6 use the high-motion windows with the largest advantage over Direct. These show the mechanism where it matters; typical behaviour is given by the aggregate results, and the galleries below use a random selection. In Figure 1(b) every model’s own step-10 features (and the true ones) are decoded to RGB by the feature-to-RGB decoder of Appendix D.4, shown at the camera’s aspect ratio and cropped to the same arm region (top-left 11×11 patches); the corner numbers are feature errors of the whole frame. In Figure 1(c), Δ error compares SHIFTWM with Direct (the strongest learned baseline) on DROID and Hamlyn, and with the same published model without the head for V-JEPA 2-AC and DINO-WM; skill is the share of persistence error removed.

Largest-advantage windows per benchmark. For every forecasting benchmark with finished SHIFTWM, Direct and AR runs (lowest common seed), the next figures show three held-out windows chosen by a fixed rule: over all test windows (stride 2) we compute each method’s step-10 feature MSE and the true change $\|Z_{t+10} - Z_t\|^2$; among windows in the top 50% of true change we rank by the gain $1 - \text{err}_{\text{SHIFTWM}} / \min(\text{err}_{\text{Direct}}, \text{err}_{\text{AR}})$ and keep the top three from distinct episodes. Every method is shown on the same window and frame, with one error scale per dataset; decoded forecasts use the decoder of Appendix D.4. These are selected best-case examples, not typical ones: the random galleries follow, and aggregate results are in the tables.

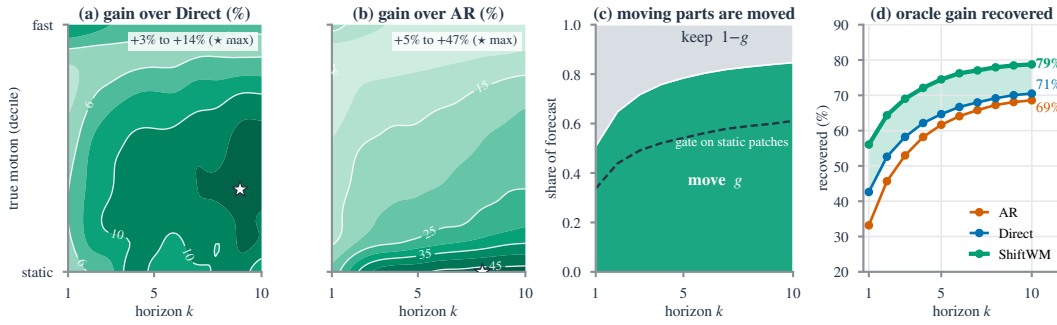


Figure 13: **Anatomy of the gains (held-out DROID).** (a,b) Error reduction over horizon and motion decile. (c) Gate on moving (area) and static (dashed) patches. (d) Share of the oracle-move gain recovered on moving patches.



Figure 14: **Transport vs. optical flow.**

E.4 DISCUSSION

When transport helps. The gains are largest where content moves within the transport window and keeps its appearance. On DROID, SHIFTWM has 7.2% lower moving-patch error than Direct, and its advantage holds in every decile of motion (+5.1% to +13.2%). On the surgical tasks the instruments move less within ten steps, the gains are small (3.6% vs. Direct), and the segmentation view does not separate the learned models (Appendix E.2).

When it does not. As a plug-in to DINO-WM on Wall, where the backbone conditions on a single history frame, the head raises latent error by 18.4%. With one history frame, after the first rollout step the head transports DINO-WM’s own predictions rather than measured features (Appendix B.3); we have not yet tested this explanation, and the $S=1$ ablation in Table 3 addresses it. Content that is not visible in any observed frame can only come from the correction (Figure 21), and motion larger than the window, such as camera ego-motion, lies outside the transport prior; we removed the rare RT-1 episodes in which the base moves.

Gate versus transport. The test-time gate knockout shows that the trained model relies on transport on moving patches (56.8% higher error without it, against 1.8% on static patches); it is not a retrained control, and the controlled comparison is Direct (7.2% lower moving-patch error). On static patches the mean gate still rises to about 0.6 at $k=10$ (Figure 13c), so static content is also routed through transport; whether that transport is close to a copy through the identity bias is not yet measured. On static patches, part of the advantage may come from the gated identity path alone; separating the two requires a gated regression head without transport, which we list as future work.

Uncertainty. Under uncertain motion, the MSE-optimal forecast averages over plausible sources (Proposition 1). SHIFTWM averages measured features rather than regressing a mean, but it remains

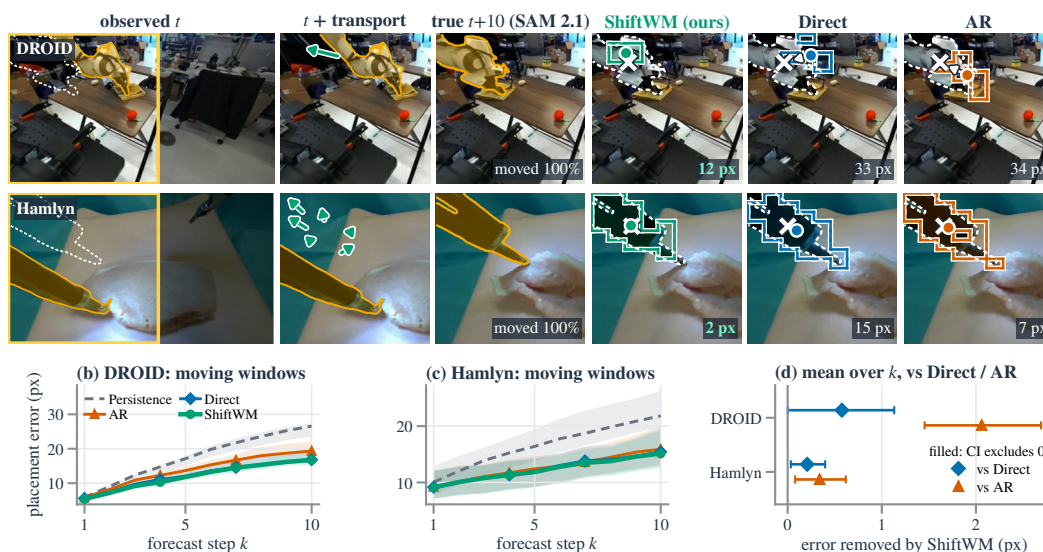


Figure 15: **Where does each model put the arm?** (a) Predicted arm region (outline), its centroid (dot) and the true centroid ($\times$) on the same true frame. (b,c) Placement error vs. horizon. (d) Error removed by SHIFTWM.

Table 19: **Arm placement error** (px of the displayed frame, DROID 320×180 , Hamlyn 396×224 ; lower is better). Gains: moving windows, mean over k , paired 95% CI: **green**: CI excludes zero.

Dataset	Method	all windows		moving windows			error removed by SHIFTWm (px)	
		avg. over k	$k=K$	avg. over k	$k=5$	$k=K$	vs Direct	vs AR
DROID	Persistence	11.3	16.0	17.6	17.1	26.6		
	AR	10.1	14.0	13.9	13.7	19.3		
	Direct	8.9	12.0	12.2	12.0	16.8		
	SHIFTWm	8.7	11.7	12.0	11.8	16.7	+0.6 [+0.0, +1.1]	+2.1 [+1.5, +2.7]
Hamlyn	Persistence	10.6	13.3	16.6	16.4	21.8		
	AR	8.7	10.5	12.7	12.4	15.9		
	Direct	8.6	10.2	12.4	12.0	15.4		
	SHIFTWm	8.5	10.1	12.3	11.8	15.1	+0.2 [+0.0, +0.4]	+0.3 [+0.1, +0.6]

a deterministic model; stochastic feature forecasters (Boduljak et al., 2025; Porcher et al., 2026) target multimodal futures and are complementary.

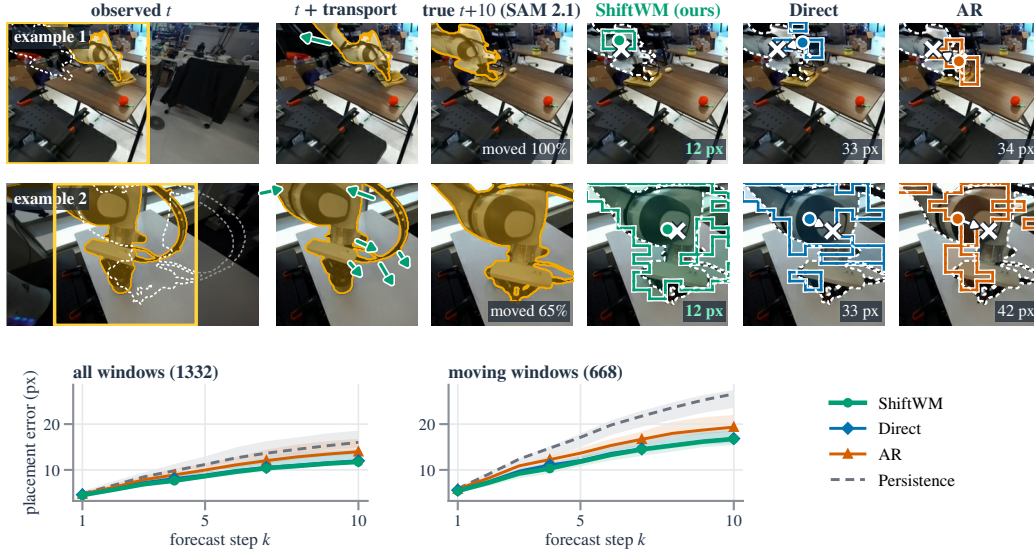


Figure 16: **Arm placement, DROID** (target: robot arm; 1332 windows, 123 episodes). Same true frame $t+10$ in every column; examples selected as in Appendix E.2. Bottom: placement error vs. horizon.

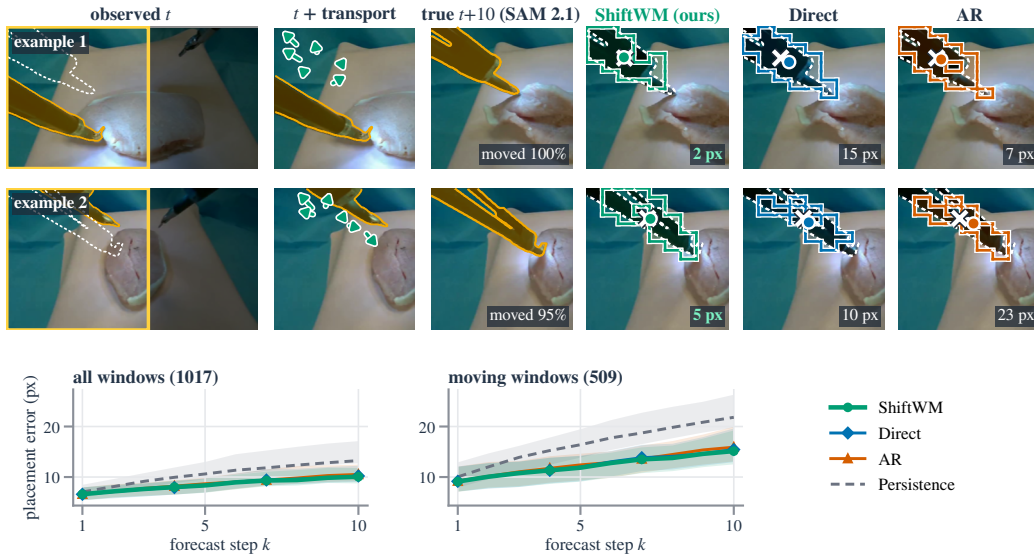


Figure 17: **Arm placement, Hamlyn** (target: instruments; 1017 windows, 140 episodes). Same true frame $t+10$ in every column; examples selected as in Appendix E.2. Bottom: placement error vs. horizon.

1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727

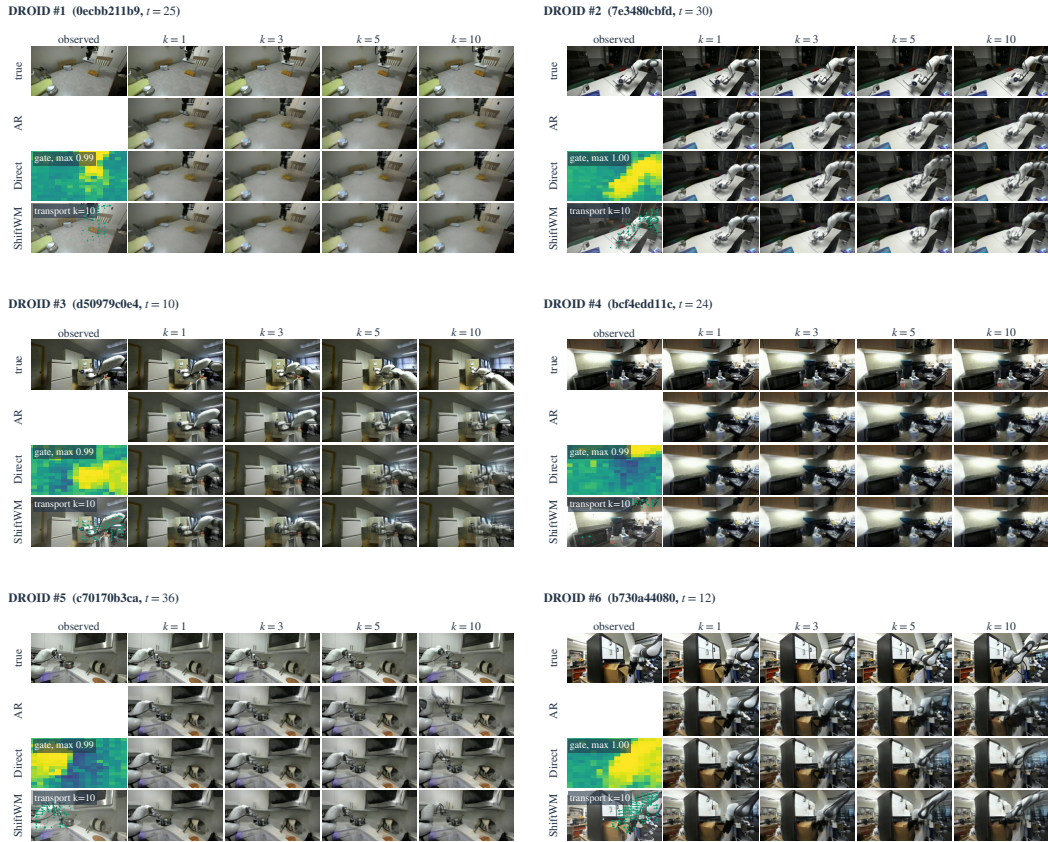
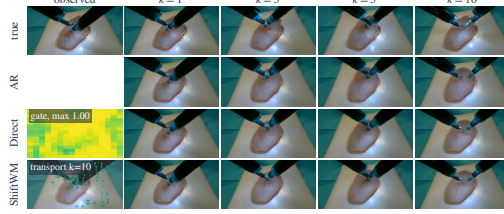
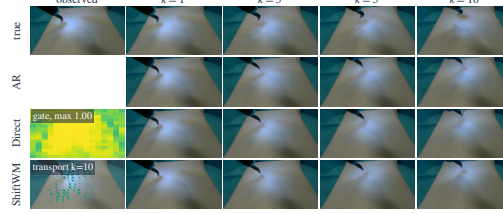


Figure 18: **DROID test rollouts (random selection).**

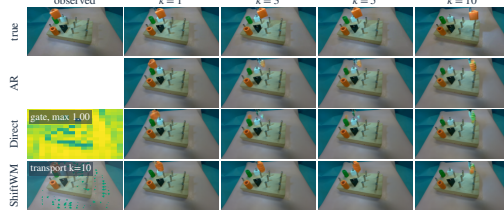
knot tying (ep 000039, $t = 22$)



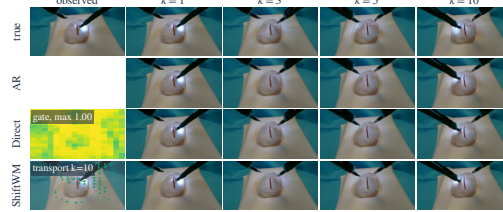
needle grasp and handover (ep 000022, $t = 31$)



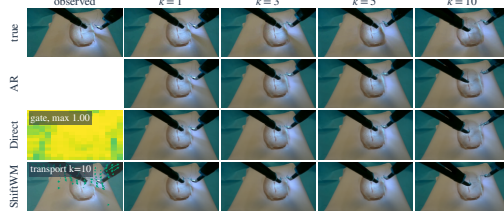
peg transfer (ep 000083, $t = 21$)



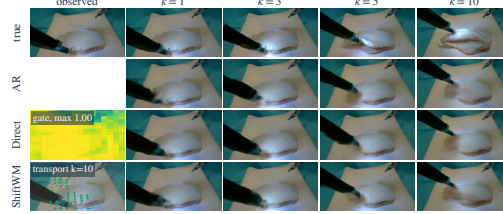
suturing 1 (ep 000042, $t = 22$)



suturing 2 (ep 000083, $t = 72$)



tissue lifting (ep 000071, $t = 9$)



tissue retraction (ep 000053, $t = 9$)

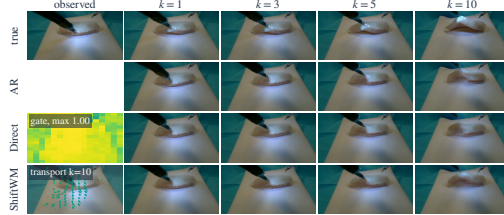
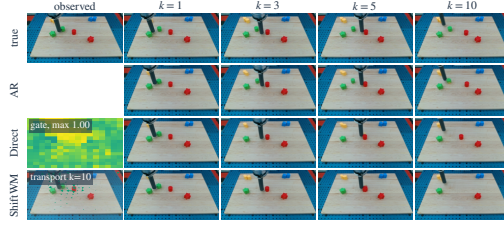
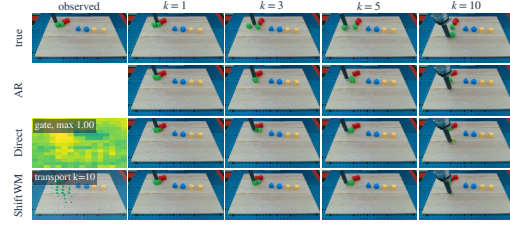


Figure 19: Surgical test rollouts (one per task, random selection).

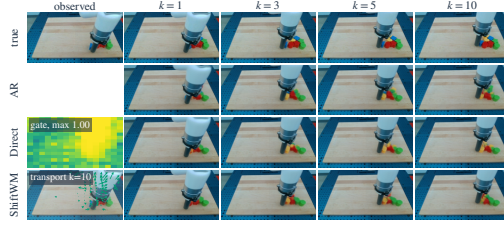
Language-Table #1 (ep train00662_312, $t = 16$)



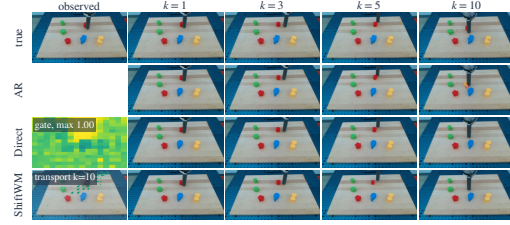
Language-Table #2 (ep train00170_396, $t = 23$)



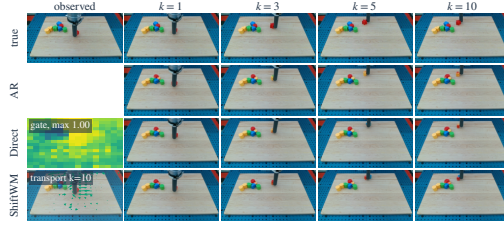
Language-Table #3 (ep train00611_310, $t = 6$)



Language-Table #4 (ep train00383_188, $t = 6$)



Language-Table #5 (ep train00406_391, $t = 21$)



Language-Table #6 (ep train00858_415, $t = 11$)

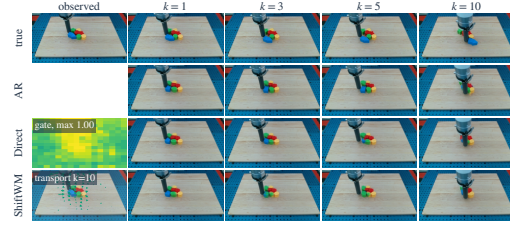


Figure 20: Language-Table test rollouts (random selection). Frames at native 16:9 aspect.

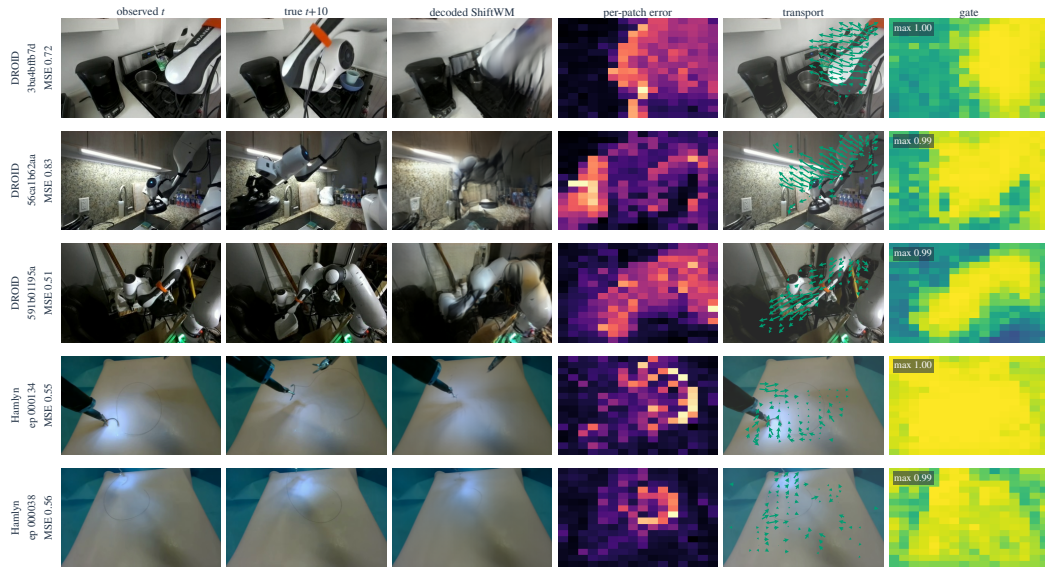


Figure 21: Failure cases.